

Mini-EDR Agent

남궁명수
2026년 6월 10일

[1] Mini-EDR 개발 계획	4
[1-1] 프로젝트 배경	4
[1-2] 개발 환경	4
[1-3] 참고 프로젝트	4
[1-4] 시스템 구조	5
[1-5] 동시성 설계	5
[2] EDR	6
[2-1] EDR이란 무엇인가?	6
[2-2] 조직에서 EDR을 사용하는 이유	7
[2-3] EDR 작동 방식	8
[2-4] EDR과의 비교	12
[3] SysmonForLinux 참고	14
[3-1] 오픈소스 분석하기	14
[1단계] 빌드 가이드 및 설정 파일 파악하기	14
[2단계] 최상위 디렉터리 구조 매핑하기	15
[3단계] 엔트리 포인트에서 출발하는 하향식(Top-Down) 흐름 추적	16
[4단계] 4대 핵심 기능별 실전 메커니즘 헌팅	17
[4] 커널 공간 및 사용자 공간 아키텍처 설계	19
[4-1] 전체 시스템 아키텍처	19
[4-2] Kernel Space 구성	20
[5] 구체화 및 적용	21
[5-1] Auditd	21
[5-2] User Space - Collector	29
[5-3] User Space - Event Dispatcher	31
[5-4] User Space - Rule Engine	32
[5-4] User Space - Alert	33
[7] Mini EDR Rule	35
[7-1] EDR Rule Engine	35

[7-2] Mini EDR Rule 설계 절차 설명	37
[7-3] Mini EDR Rule 설계	40

[1] Mini-EDR 개발 계획

[1-1] 프로젝트 배경

Mini-EDR 프로젝트는 Linux 환경에서 동작하는 경량 EDR(EndPoint Detection and Response) 에이전트를 직접 구현하기 위해 시작하였다.

프로젝트를 진행하게 된 이유는 크게 두 가지이다.

첫 번째는 Linux 시스템 프로그래밍에 대한 이해를 높이기 위함이다. 단순히 웹 애플리케이션을 개발하는 수준을 넘어 운영체제에서 발생하는 프로세서 생성, 파일 접근, 네트워크 연결과 같은 이벤트가 어떻게 발생하고 수집되는지 직접 경험해 보고자 한다.

두 번째는 Go 언어의 동시성 모델을 실제 프로젝트에 적용해 보기 위함이다.

Go의 Goroutine과 Channel은 학습만으로는 한계가 있으며, 이벤트가 지속적으로 발생하는 시스템을 직접 구현하면서 동시성 설계와 데이터 흐름을 경험하는 것이 중요하다고 판단하였다.

[1-2] 개발 환경

프로젝트는 현재 학습용으로 사용하고 있는 구형 Linux PC를 대상으로 개발한다.

하드웨어 사양은 다음과 같다.

- 삼성 매직스테이션 DM-C200-PAD1
- Intel Pentium E5400 (2.7GHz)
- DDR3 2GB Memory
- 250GB SATA HDD
- Intel GMA x4500

최신 하드웨어가 아니기 때문에 불필요하게 무거운 구조보다는 적은 메모리와 CPU 자원으로 동작할 수 있는 구조를 우선 고려한다.

[1-3] 참고 프로젝트

프로젝트를 진행하면서 Microsoft의 SysmonForLinux 오픈소스를 참고할 예정이다.

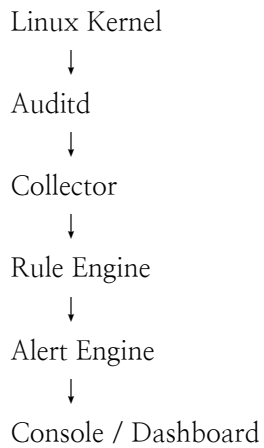
다만 SysmonForLinux를 그대로 재구현하는 것이 목적은 아니다.

SysmonForLinux를 분석하는 이유는 다음과 같다.

- 어떤 이벤트를 수집하는지 확인하기 위함
- 이벤트를 어떤 구조로 표현하는지 확인하기 위함
- 규칙 엔진이 어떤 방식으로 동작하는지 확인하기 위함
- 실제 EDR 에이전트의 구조를 이해하기 위함

[1-4] 시스템 구조

1차 버전은 다음 구조를 목표로 한다.



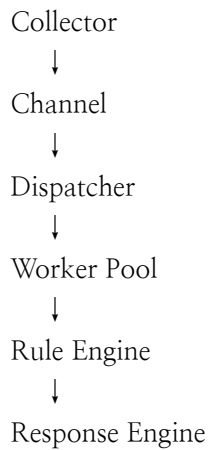
- Collector는 Auditd 이벤트를 수신하여 내부 이벤트 객체로 변환
 - Rule Engine은 이벤트를 분석하여 정의된 규칙과 비교
 - Alert Engine은 규칙에 일치하는 이벤트를 탐지했을 때 경고를 생성
- 최종적으로 이벤트와 경고 정보는 콘솔에서 확인할 수 있도록 구성할 예정이다.

[1-5] 동시성 설계

프로젝트에서 가장 중요하게 학습하고 싶은 기술 중 하나는 Go의 동시성 모델이다.

이벤트 수집, 분석, 탐지 과정을 단일 스레드에서 처리하는 대신 파이프라인 구조로 분리할 예정이다.

다음은 예시이다.



[2] EDR

[2-1] EDR이란 무엇인가?

(1) EDR(엔드포인트 탐지 및 대응)의 정의

엔드포인트 탐지 및 대응(EDR)은 안티바이러스 소프트웨어와 같은 기존의 보안 도구를 우회하는 고도화된 사이버 위협으로부터 조직의 자산을 보호하는 보안 소프트웨어이다.

실시간 분석과 AI 기반 자동화를 활용하여 최종 사용자, 엔드포인트 장치, IT 자산을 안전하게 방어한다.

(2) 데이터 수집 대상(엔드포인트)

EDR은 네트워크에 연결된 모든 엔드포인트 기기에서 지속적으로 데이터를 수집한다.

주요 대상은 다음과 같다.

- 개인용 기기: 데스크톱 및 노트북
- 인프라 장비: 서버
- 휴대용 기기: 모바일 디바이스
- 기타 연결 장치: IoT(사물 인터넷) 디바이스

(3) EDR의 핵심 기능 및 자동 방식

실시간 데이터 분석

수집된 모든 엔드포인트 데이터를 실시간으로 분석한다. 이를 통해 이미 알려진 위협뿐만 아니라, 정상적인 패턴을 벗어난 의심스러운 사이버 위협의 징후까지 정밀하게 찾아낸다.

자동화된 대응

식별된 위협에 대해 시스템이 자동으로 대응한다. 위협을 초기에 차단함으로써 보안 사고로 인한 피해를 사전에 예방하거나 발생할 수 있는 피해 규모를 최소화한다.

[2-2] 조직에서 EDR을 사용하는 이유

(1) 엔드포인트 디바이스의 보안 취약성

오늘날 성공적인 사이버 공격의 90%, 그리고 실제 데이터 침해 사고의 70%가 엔드포인트 디바이스(사용자 기기, 서버 등)에서 시작된다. 이 때문에 엔드포인트 보안은 기업의 전체 보안 체계에서 가장 중요한 요소로 꼽힌다.

(2) 기존 보안 솔루션의 한계

안티바이러스, 안티 멀웨어, 방화벽 등 기존의 엔드포인트 보안 솔루션은 시간이 지나며 발전해 왔으나, 여전히 ‘알려진 파일 기반’ 또는 ‘시그니처(서명) 기반’의 위협을 탐지하는 데 머물러 있다는 명백한 한계가 있다.

- 소셜 엔지니어링 공격 차단 어려움

민감한 데이터를 유출하게 만들거나 악성 코드가 숨겨진 가짜 웹 사이트로 유인하는 ‘피싱(Phishing)’ 메시지 같은 공격을 차단하는 데 매우 취약하다. 특히 피싱은 랜섬웨어를 유포하는 가장 일반적인 경로이다.

- 파일리스(Fileless) 공격에 무력화

최근 급증하는 ‘파일리스 사이버 공격’은 파일이나 서명 스캐닝 시스템을 완벽히 피하기 위해 컴퓨터의 메모리 내에서만 작동한다. 기존 솔루션으로는 이러한 공격을 감지할 수 없다.

- 잠복 중인 지능형 위협 탐지 불가

기존 도구는 감시망을 교묘히 빠져나가는 지능형 위협을 무력화하지 못한다. 이러한 위협들은 랜섬웨어 공격이나 제로데이 익스플로잇(보안 취약점이 패치되기 전 공격) 등 대규모 공격을 감행하기 위해, 수개월 동안 네트워크에 잠복하여 데이터를 수집하고 취약점을 파악한다.

(3) 조직이 EDR을 반드시 도입해야 하는 이유

EDR은 앞서 언급한 기존 엔드포인트 보안 솔루션들의 치명적인 한계점들을 극복한다.

- 피해 발생 전 실시간 위협 식별 및 억제

강력한 위협 탐지 분석과 자동 대응 기능을 통해, 대개 사람(보안 담당자)의 개입 없이도 네트워크 경계를 뚫고 들어온 잠재적 위협을 초기에 식별하고 격리한다.

- 보안팀을 위한 선제적 대응 도구 제공

이미 발생한 위협에 대응하는 것을 넘어, 보안팀이 의심스러운 징후나 알려지지 않은 새로운 위협을 스스로 발견하고, 조사하며, 사전에 예방할 수 있는 고도화된 도구를 제공한다.

[2-3] EDR 작동 방식

(1) EDR의 5가지 핵심 기능

공급 업체마다 세부적인 차이는 있지만, EDR 솔루션은 일반적으로 다음의 5가지 핵심 기능을 결합하여 작동한다.

1. 지속적인 엔드포인트 데이터 수집
2. 실시간 분석 및 위협 탐지
3. 자동화된 위협 대응
4. 위협 격리 및 해결
5. 위협 헌팅 지원

[기능 1] 지속적인 엔드포인트 데이터 수집

(1) 수집 대상 데이터

EDR은 네트워크에 연결된 모든 엔드포인트 디바이스에서 다음과 같은 데이터를 중단 없이 지속적으로 수집한다.

- 프로세스 및 성능 상태, 시스템 구성 변경 사항
- 네트워크 연결 이력, 파일 및 데이터의 다운로드/전송기록
- 최종 사용자 또는 디바이스의 세부 동작 정보

(2) 데이터 수집 및 저장 방식

- 저장 위치

수집된 데이터는 일반적으로 클라우드 기반의 중앙 데이터베이스 또는 데이터 레이크(Data Lake)에 안전하게 저장된다.

- 수집 방법

대부분의 EDR 솔루션은 각 엔드포인트 디바이스에 가볍게 작동하는 경량 데이터 수집 도구(에이전트)를 설치하여 데이터를 수집한다. 일부 솔루션은 디바이스 자체 운영 체제(OS)의 내장 기능에 의존하기도 한다.

[기능 2] 실시간 분석 및 위협 탐지

EDR은 고급 기법과 머신 러닝 알고리즘을 활용하여 수집된 데이터를 실시간으로 들여다보고, 알려진 위협이나 의심스러운 패턴을 식별한다.

(1) 탐지하는 두 가지 핵심 지표

- 침해 지표(IOC, Indicators of Compromise)
이미 발생한 잠재적 공격이나 침해 흔적과 일치하는 행위 및 이벤트

- 공격 지표(IOA, Indicators of Attack)
알려진 사이버 위협이나 사이버 범죄자가 공격을 시도할 때 나타나는 행위 및 이벤트

(2) 위협 판단을 위한 데이터 상호 연관 분석

EDR은 수집한 엔드포인트 데이터를 외부의 위협 인텔리전스 서비스와 실시간으로 대조한다.

이를 통해 새로운 사이버 위협의 전술, 악용되는 인프라 취약성 등 지속적으로 업데이트되는 최신 정보를 반영한다.

- 위협 인텔리전스 종류

EDR 공급업체의 자체 서비스, 제3자(타사) 서비스, 또는 커뮤니티 기반 서비스 등

- 글로벌 지식 기반 활용

많은 EDR 솔루션이 미국 정부 등이 참여하여 해커의 공격 전술과 기술을 정립해둔 글로벌 지식 기반인 Mitre ATT&CK에 데이터를 매핑하여 분석

(3) 독자적인 조사 및 노이즈 필터링

- 기록 및 기준점 비교

실시간 데이터를 과거의 기록 데이터 및 정상 상태의 기준점과 비교하여, 비정상적인 사용자 활동이나 사이버 보안 인시던트 징후를 스스로 찾아낸다.

- 오탐(False Positive) 제거

진짜 위험한 ‘신호’와 정상적인 활동이 오인된 ‘노이즈’를 명확히 구분해 줌으로써, 보안 분석가가 진짜 중요한 인시던트에만 집중할 수 있도록 돕는다.

(4) SIEM(보안 정보 및 이벤트 관리) 솔루션과의 통합

많은 기업이 애플리케이션, 데이터베이스, 웹 브라우저, 네트워크 하드웨어 등 IT 인프라 전 계층의 보안 정보를 모으는 SIME과 EDR을 연동한다.

SIME이 제공하는 광범위한 컨텍스트(맥락)는 EDR의 위협 식별, 우선 순위 지정, 조사 및 수정 분석 능력을 더욱 강하게 만든다.

(5) 중앙 관리 콘솔을 통한 가시성 확보

EDR은 이 모든 중요한 데이터와 분석 결과를 사용자 인터페이스(UI) 역할을 하는

중앙 관리 콘솔에 직관적으로 요약해준다.

보안 팀은 이 콘솔을 통해 전사 엔드포인트의 보안 상태를 완벽하게 모니터링(가시성 확보)하고, 즉각적인 조사와 위협 대응을 시작할 수 있다.

[기능 3] 자동화된 위협 대응

자동화는 EDR의 핵심인 ‘신속한 대응’을 가능하게 하는 요소이다. 보안팀이 사전에 정의한 규칙이나 머신러닝 알고리즘이 학습한 기준을 바탕으로, EDR은 사람의 개입 없이 다음과 같은 작업을 자동으로 수행한다.

(1) 알림 및 분류 자동화

- 보안 분석가에게 특정 위협 또는 의심스러운 활동 즉시 알림
- 위협의 심각도에 따라 경고의 우선순위를 분류 및 지정

(2) 가시성 및 추적력 제공

- 인시던트나 위협의 네트워크 유입 및 중단 시점을 근본 원인까지 추적하는 ‘추적 보고서’ 생성

(3) 즉각적인 위협 차단 행위

- 위협이 감지된 엔드포인트 디바이스의 연결을 끊거나 최종 사용자를 네트워크에서 강제 로그오프
- 악성 시스템 또는 엔드포인트 프로세스를 즉시 강제 종료
- 엔드포인트가 악성 파일이나 의심스러운 이메일 첨부 파일을 실행(폭발)하지 못하도록 원천 차단

(4) 전사 확산 방지

- 안티바이러스 또는 악성코드 방지 소프트웨어를 트리거하여, 네트워크 내의 다른 엔드포인트에게도 동일한 위협이 존재하지 않는지 즉시 교차 검사

(5) 외부 시스템(SOAR)과의 통합

EDR은 자체적인 위협 조사 및 해결을 자동화할 뿐만 아니라, SOAR(보안 오케스트레이션, 자동화 및 대응) 시스템과 통합될 수 있다. 이를 통해 다른 보안 도구들과 연계된 종합 보안 대응 플레이북(인시던트 대응 시퀀스)을 자동화한다.

- 도입 효과
보안 팀이 인시던트에 더 빠르게 대응하여 피해를 최소화할 수 있으며, 한정된 자원과 인력으로 최대한의 효율성을 발휘하도록 지원한다.

[기능 4] 조사 및 해결

위협이 성공적으로 격리되면, EDR은 보안 분석가가 위협의 실체를 명확히 규명하고 해결할 수 있는 도구를 지원한다.

(1) 정밀 포렌식 분석

보안 분석가는 EDR의 포렌식 기능을 활용하여 다음과 같은 상세 정보를 파악한다.

- 위협의 최초 유입 경로 및 근본 원인 규명
- 공격으로 인해 영향을 받은 파일들의 범위 식별
- 공격자가 네트워크 내부로 진입하고 이동(수평 이동)하여 인증 자격 증명에 접근할 수 있었던 보안 취약점 식별

(2) 해결(치유) 작업 수행

분석가는 수집된 정보를 기반으로 EDR의 해결 도구를 사용하여 위협을 완전히 제거한다.

- 악성 자산 제거: 악성 파일 삭제 및 엔드포인트 내 잔재 제거
- 시스템 정상화: 손상된 시스템 구성, 레지스트리 설정, 데이터 및 애플리케이션 파일 복원
- 취약점 보완: 동일 취약점을 악용한 재공격 방지를 위한 업데이트 및 패치 적용
- 보안 고도화: 향후 재발 방지를 위한 EDR 내부 탐지 규칙 업데이트

[기능 5] 위협 헌팅 지원

위협 헌팅은 자동화된 보안 도구들이 아직 감지하지 못했거나, 네트워크 내에 알려지지 않은 채 숨어 있는 위협을 보안 분석가가 선제적으로 찾아내는 ‘사전 예방적’ 보안 활동이다.

지능형 위협은 대규모 침해를 일으키기 전 수개월 동안 잠복하며 정보를 수집하므로,

시기적절한 위협 헌팅은 탐지 및 해결 시간을 단축하고 피해를 원천 차단하는 데 필수적이다.

(1) EDR 데이터 및 기능의 재활용

위협 헌터들은 EDR이 위협 탐지 및 수정에 사용하는 것과 동일한 데이터 소스, 고급 분석, 자동화 기능에 의존하여 조사한다.

- 추적 및 연계 분석

포렌식 분석 데이터나 특정 공격 그룹의 공격 방법을 설명하는 MITRE ATT&CK 데이터를 기반으로, 네트워크 내에 숨겨진 특정 파일, 비정상적 구성 변경, 기타 공격 흔적(아티팩트)을 역추적한다.

(2) 헌팅 전용 도구 및 인터페이스 제공

EDR은 분석가가 위협 헌팅을 원활하게 수행할 수 있도록 사용자 인터페이스(UI) 및 프로그래밍 방식의 접근을 모두 지원한다.

- 상시 쿼리 및 분석

임시 검색, 대용량 데이터 쿼리, 위협 인텔리전스와의 상관관계 분석 지원

- 생산성 도구 포함

반복적인 조사 작업을 자동화하기 위한 간단한 스크립팅 언어부터, 복잡한 명령어 없이도 데이터를 찾을 수 있는 ‘자연어 쿼리 도구’까지 다양한 편의 기능을 제공한다.

[2-4] EDR과의 비교

(1) EDR vs EPP(엔드포인트 보호 플랫폼)

EPP와 EDR은 모두 ‘엔드포인트(기기)’를 보호한다는 공통점이 있지만, 주요 목적과 위협을 대하는 방식에서 명확한 차이가 있다.

- EPP(Endpoint Protection Platform)

차세대 안티바이러스(NGAV), 안티멀웨어, 웹 필터, 방화벽, 이메일 게이트웨이 등을 하나로 결합한 통합 보안 플랫폼이다.

주로 ‘알려진 위협’이나 알려진 패턴으로 움직이는 공격을 사전에 방지하는데 초점을 맞춘다.

- EDR(Endpoint Detection and Response)

EPP와 같은 기존 보안 기술을 우회하여 들어온 ‘알려지지 않은 위협’이나 잠재적 위협을 실시간으로 식별하고 차단한다.

- 최근 추세

많은 EPP 솔루션이 고도화되면서 지능형 위협 탐지 분석, 사용자 행위 분석 등 EDR의 기능을 포괄하는 형태로 진화하고 있다.

(2) EDR vs XDR(확장 탐지 및 대응)

XDR은 분석 및 AI 기반 위협 탐지라는 점에서 EDR과 유사하지만, 보호하는 ‘범위(Scope)’가 훨씬 넓다.

- EDR의 범위

‘엔드포인트(디바이스)’ 단에 집중하여 위협을 탐지하고 대응한다.

- XDR(Extended Detection and Response)의 범위

엔드포인트를 넘어 네트워크, 이메일, 애플리케이션, 클라우드 워크로드 등 조직의 하이브리드 인프라 전체로 보안 도구를 확장 및 통합한다.

- XDR의 특징 및 효과

SIME, SOAR 등 전사 사이버 보안 기술을 통합하여 각 보안 제어 지점과 원격 측정 데이터를 단일 중앙 시스템으로 묶어준다.

이를 통해 모든 도구가 상호 유기적으로 조정되므로, 업무가 과중한 보안운영센터(SOC)의 효율성과 효과성을 극대화할 수 있는 차세대 기술이다.

(3) EDR vs MDR(관리형 탐지 및 대응)

MDR은 기술적인 솔루션이라기보다, 보안을 운영하는 ‘제공 방식(서비스)’의 차이이다.

- EDR/XDR

조직이 직접 도입하여 내부 보안팀이 운영하는 ‘소프트웨어/기술’이다.

- MDR(Managed Detection and Response)

보안 운영을 외부에 위탁하는 ‘아웃소싱 사이버 보안 서비스’이다.

- MDR의 특징 및 효과

MDR 공급업체는 대개 클라우드 기반의 EDR 또는 XDR 기술을 활용하며, 고도로 숙련된 자체 보안 분석가 팀을 통해 연중 무휴(24/7) 실시간 모니터링, 탐지 및 해결 서비스를 원격으로 제공한다. 내부 보안 전문 인력이 부족하거나, 고가의 보안 기술을 직접 구축하기에 예산이 한정된 조직에게 매우 매력적인 대안이다.

[3] SysmonForLinux 참고

[3-1] 오픈소스 분석하기

1단계: 빌드 가이드 및 설정 파일(*.md, *.xml) 파악하기
2단계: 최상위 디렉터리 구조 매핑하기
3단계: 엔트리 포인트(Entry Point, main())부터 흐름 추적하기
4단계: 4대 핵심 기능별 “키워드 헌팅”

[1단계] 빌드 가이드 및 설정 파일 파악하기

오픈소스 프로젝트를 분석할 때는 소스코드를 바로 읽기보다 README와 설정 파일(XML, YAML, JSON 등)을 먼저 확인하는 것이 효율적이다. 설정 파일은 프로그램이 어떤 데이터를 수집하고 어떤 규칙을 적용하는지를 가장 직관적으로 보여주기 때문이다.

SysmonForLinux 저장소의 test/ruletest/configs 디렉터리에서 여러 XML 규칙 파일들을 확인하였다. 파일명에 포함된 basic_include, all_conditions, multiple_event_types, file_and_network, all_event_types 등을 통해 Sysmon이 이벤트 기반으로 동작하며 다양한 시스템 이벤트를 수집 및 필터링하는 프로그램임을 추측할 수 있다.

예를 들어, 01_basic_include_xml에는 다음과 같은 규칙이 존재한다.

```
<Sysmon schemaversion="4.90">
<EventFiltering>
  <RuleGroup name="" groupRelation="or">
    <ProcessCreate onmatch="include">
      <Image condition="is">/usr/bin/ls</Image>
    </ProcessCreate>
  </RuleGroup>
</EventFiltering>
</Sysmon>
```

이 규칙은 프로세스 생성(ProcessCreate) 이벤트가 발생했을 때 실행 파일 경로(Image)가 “usr/bin/ls”와 정확히 일치(is)하는 경우 해당 이벤트를 수집 대상으로 포함(include)하라는 의미이다.

더 나아가 23_all_event_types.xml 파일을 종합적으로 분석한 결과, SysmonForLinux는 다음과 같은 핵심 이벤트 유형들을 지원함을 확인하였다.

- ProcessCreate_Rules (프로세스 생성 감시)
- ProcessTerminate_Rules (프로세스 종료 감시)
- NetworkConnect_Rules (네트워크 연결 감시)
- FileCreate_Rules / FileDelete_Rules (파일 생성 및 삭제 감시)

XML 규칙 파일 분석을 통해 “Sysmon은 프로세스, 파일, 네트워크 등 운영체제 전반의 이벤트를 특정 필터 조건에 따라 필터링한다”는 목적을 파악했다. 다음 단계에서는 이 XML 규칙들이 실제 소스코드 상에서 어떻게 인젝되고 커널로 전달되는지 구조적 연결고리를 추적한다.

[2단계] 최상위 디렉터리 구조 매핑하기

AI와 tree -L 2 명령어를 통해 최상위 디렉터리 구조 및 파일명을 기반으로 SysmonForLinux 내부의 거시적인 데이터 흐름과 책임 영역을 역공학 방식으로 추론하여 기록한다.

SysmonForLinux 추론 데이터 파이프라인

Linux Kernel

↓ (시스템콜 발생)

eBPF Programs (ebpfKern) : 커널 공간에서 이벤트 1차 필터링 및 수집

↓ (Ring Buffer / Event Transfer)

User Space Processing (sysmonforlinux) : 사용자 공간의 핵심 데몬 프로세스로 데이터 이관

↓

Rule Engine (linuxRules) : XML 규칙 기반 필터링 및 탐지 매칭

↓

XML Output (outputxml) : 탐지된 이벤트를 표준 포맷으로 강제

↓

Log Storage / Viewer : 시스템 로그(syslog) 저장 및 관리자 가시성 제공

디렉터리 구조 기반 핵심 인사이트

- 커널과 사용자 공간의 명확한 분리

커널 영역(ebpfKern)은 최소한의 시스템 콜 이벤트만 안전하게 수집하고, 무거운 규칙 매칭(linuxRules)과 로그 정제(outputxml)는 사용자 공간 데몬 프로세스에서 비동기적으로 처리하는 구조적 효율성을 배운다.

- mini-edr 도입 계획

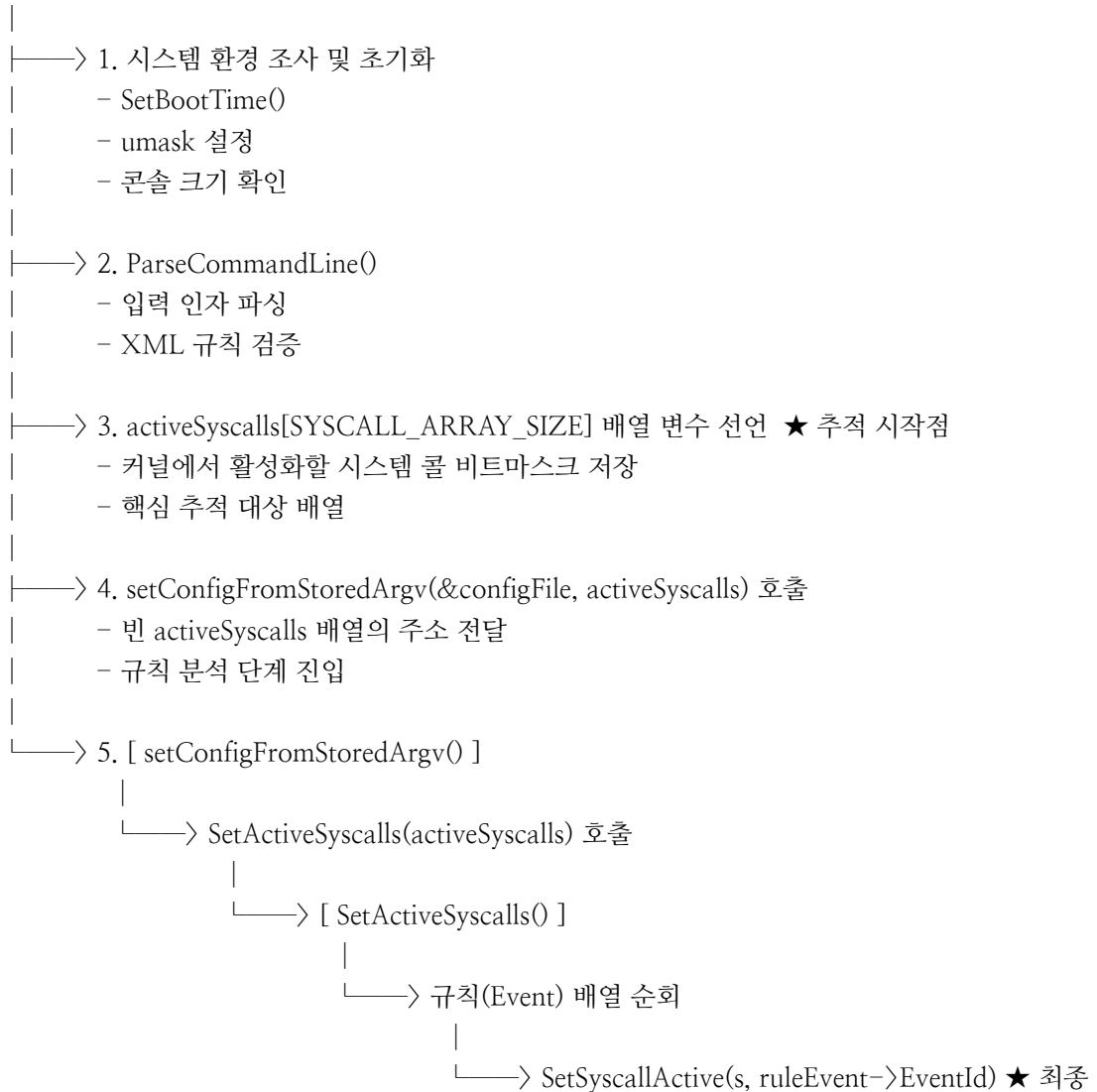
프로젝트 역시 수집 레이어와 사용자 공간 레이어를 분리하고, Go의 동시성 모델을 통해 커널이 밀어주는 데이터를 사용자 공간에서 안전하게 이관 및 분석하는 구조적 기틀을 확보한다.

[3단계] 엔트리 포인트에서 출발하는 하향식(Top-Down) 흐름 추적

핵심 소스코드 파일인 SysmonForLinux.c 분석을 통해, 진입점인 main() 함수에서 출발하여 데이터 필터링 규칙이 생성되는 전 과정을 변수와 함수의 인과관계에 따라 하향식으로 추적하였다..

(1) 메인 프로세스 제어 흐름 및 아키텍처 매핑

[main() 진입]



[4단계] 4대 핵심 기능별 실전 메커니즘 헌팅

3단계에서 수집된 SysmonForLinux.c 내부의 실전 파이프라인 함수들과 이벤트 핸들러 구조를 기반으로, mini-edr 설계에 직접적으로 이식할 4대 핵심 기능별 물리적 구현체와 데이터 모델을 매핑한다.

(1) 수집 & 매핑

- 실전 구현체 단서: SetSyscallActive() 함수 내부의 switch-case 메커니즘

```
void SetSyscallActive(bool *s, ULONG eventId) {
    switch(eventId) {
        case SYSMONEVENT_CREATE_PROCESS_EVENT_value:
            s[__NR_execve] = true; s[__NR_execveat] = true; break;
        case SYSMONEVENT_FILE_CREATE_EVENT_value:
            s[__NR_open] = true; s[__NR_creat] = true; break;
    }
}
```

- 활용 계획

Sysmon이 SYSMONEVENT_...라는 추상 규칙을 커널 시스템 콜 번호(__NR_execve 등) 배열의 true 비트로 매핑하는 방식을 100% 벤치마킹한다.

1차 목표인 AutidCollector 개발 시, 사용자 프로세스 감시 정책을 켜면 Go 에이전트 내부 매핑 테이블에 의해 auditctl -a always, exit -S execve 환경 설정 명령을 동적으로 생성 및 커널에 등록하도록 구현한다.

(2) 탐지 & 라우팅

- 실전 구현체 단서: main() 하단에서 커널 비동기 데이터 수신 콜백으로 등록된 handleEvent() 함수

```
static void handleEvent(void *ctx, int cpu, void *data, uint32_t size) {
    PSYSMON_EVENT_HEADER eventHdr = (PSYSMON_EVENT_HEADER)data;
    switch ((DWORD)eventHdr->m_EventType) {
        case ProcessCreate: processProcessCreate(eventHdr); break;
        case LinuxFileOpen: processFileOpen(eventHdr); break;
        case LinuxNetworkEvent: processNetworkEvent(eventHdr); break;
    }
}
```

- 활용 계획

커널이 던져준 이벤트 헤더의 타입(m_EventType)을 보고 switch-case 문으로 쪼개어 전용 분석 로직으로 라우팅하는 구조를 이식한다. Go 동시성 모델에서 Event Dispatcher 고루틴이 채널을 통해 낚는 로그 스트림을 받으면, 이벤트 종류를 식별하여 알맞은 Worker Pool(프로세스/파일/네트워크) 전용 채널로 안전하게 데이터를 분배하도록 설계한다.

(3) 데이터 가공 및 보고

- 실전 구현체 단서: processProcessCreate() 내부의 데이터 패키징 및 DispatchEvent() 파이프라인
- 구조적 메커니즘: 원시 커널 데이터를 전달받은 전용 함수들이 내부에서 m_ProcessId, m_ParentProcessId, PC_CommandLine 등 EDR 가시성에 필수적인 고유 필드들을 바이트 단위로 촘촘하게 패키징하여 정형 구조체(newEvent)로 가공한 뒤, DispatchEvent()를 호출하여 최종 표준 로그 (Syslog) 출력 엔진으로 밀어낸다.
- 활용 계획
auditd가 뱉어내는 파편화된 문자열에서 우리 에이전트에 필요한 PID, PPID, 명령행 인자 정보만 정규식 스트리밍 파싱으로 추출하여 정형화된 Go 구조체(Event struct)를 채우고, Alert Engine이 최종적으로 대시보드 전용 JSON 로그 레이아웃을 생성하도록 데이터 모델을 빌드한다.

(4) 대응 & 통제

- 실전 구현체 단서: main() 분기문 내부의 killOtherSysmon(), stopSysmonService() 메커니즘
- 구조적 메커니즘: 에이전트 자체의 안전한 라이프사이클 관리와 인스턴스 중복 실행 제어를 위해, 리눅스 표준 프로세스 관리 환경 및 시그널 인터페이스를 호출하여 프로세스를 통제한다.
- 활용 계획
탐지 규칙에 위배되는 명백한 유해 행위(예: 웹 브라우저 프로세스가 임시 디렉터리 내의 실행 파일을 강제 기동하는 행위) 발견 시, 해당 유해 프로세스의 PID를 식별하여 즉각 무력화 조치를 취하는 Response Engine(os.FindProcess 및 Kill)의 안전한 시그널 제어 로직 설계 근거로 삼는다.

[4] 커널 공간 및 사용자 공간 아키텍처 설계

프로젝트는 Linux Audit Framework(Auditd)를 활용하여 커널에서 발생하는 보안 이벤트를 수집하고, 사용자 공간의 Go 기반 EDR 에이전트에서 이를 분석 및 대응하는 구조로 설계하였다.

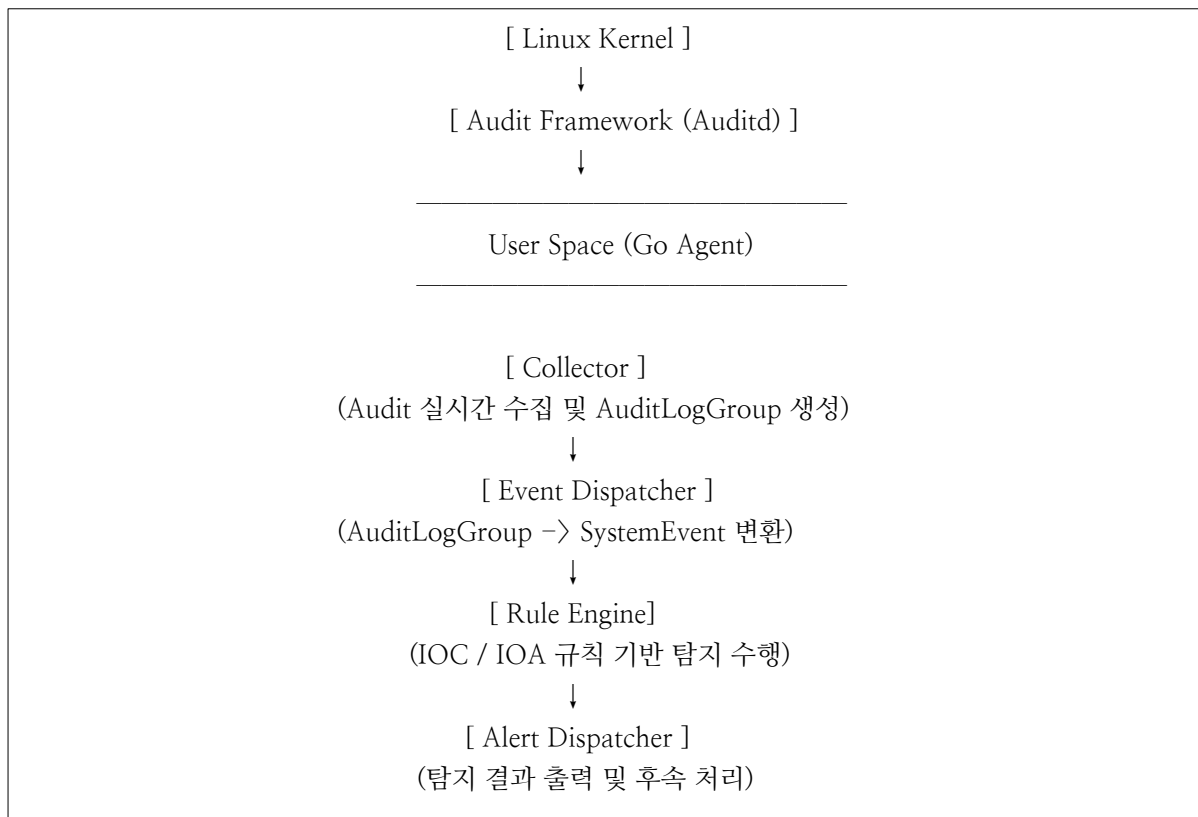
특히 대상 환경은 RAM 2GB, HDD 기반의 저사양 시스템이므로 CPU 및 디스크 I/O 사용량을 최소화하기 위해 Go 언어의 Goroutine과 Channel을 활용한 비동기 이벤트 처리 구조를 적용하였다.

[4-1] 전체 시스템 아키텍처

Linux Kernel에서 발생한 시스템 이벤트는 Audit Framework를 통해 수집되며, 이후 사용자 공간의 EDR 에이전트로 전달된다.

사용자 공간의 EDR 에이전트는 수집(Collector), 이벤트 변환(Dispatch), 탐지(Detection), 결과 처리(Alert)를 각각 독립적인 계층으로 분리하여 처리하도록 설계하였다.

전체 이벤트 처리 흐름은 다음과 같다.



각 계층은 Channel을 통해 데이터를 전달하며, Goroutine 기반의 비동기 구조를 사용하여 로그 수집과 이벤트 분석이 서로 영향을 주지 않도록 설계하였다.

[4-2] Kernel Space 구성

Kernel Space는 시스템에서 발생하는 보안 이벤트를 생성하는 계층이다.

Linux Kernel 내부에서 발생하는 프로세스 생성, 파일 생성 및 삭제, 네트워크 연결 구파일 접근, 권한 변경 등의 다양한 시스템 호출(System Call)이 발생한다. Audit SubSystem은 사전에 등록된 Audit Rule과 시스템 호출을 비교하여, 규칙과 일치하는 이벤트만 Audit Record 형태로 생성한다.

생성된 Audit Record는 Auditd에 의해 /var/log/audit/audit.log 파일에 기록되며, 이후 사용자 공간의 Mini EDR Agent가 이를 실시간으로 수집한다.

프로젝트에서는 커널 모듈이나 eBPF를 직접 개발하지 않고, Linux에서 기본적으로 제공하는 Audit Framework를 활용하여 보안 이벤트를 수집하는 방식을 채택하였다.

[5] 구체화 및 적용

[5-1] Auditd

Linux Audit Framework(Auditd)를 활용하여 커널(Kernel)에서 발생하는 보안 이벤트를 수집하고, 사용자 공간(User Space)의 Go 기반 EDR 에이전트에서 이를 분석 및 대응하는 구조로 설계하였다.

Auditd는 Linux 운영체제에서 제공하는 보안 감사 시스템으로, 커널 공간의 Audit Subsystem과 연동하여 시스템에서 발생하는 다양한 보안 관련 이벤트를 기록한다. 이를 통해 프로세스 생성 및 종료, 파일 생성 및 삭제, 네트워크 연결, 권한 상승, 민감한 파일 접근 등의 행위를 추적할 수 있다.

Auditd는 사용자 공간의 응용프로그램이 아닌 커널에서 시스템 호출(System Call)을 기반으로 이벤트를 생성하기 때문에 사용자 프로그램의 우회가 어렵고 높은 신뢰성을 제공한다.
생성된 이벤트는 /var/log/auditd/audit.log에 Audit Record 형태로 기록되며, Go 기반 EDR 에이전트는 해당 로그를 실시간으로 수집하여 이벤트를 분석한다.

수집된 Audit Record는 Collector에서 동일한 Audit Event ID(serial)를 기준으로 하나의 이벤트로 조립된 후, Event Dispatcher를 통해 Rule Engine에서 사용할 수 있는 표준 이벤트 객체로 변환된다.
이후 Rule Engine은 미리 정의된 탐지 규칙과 비교하여 IOC 및 IOA 기반의 행위 분석을 수행하고, 탐지 결과를 Alert Dispatcher로 전달한다.

Linux는 수백 개 이상의 시스템 호출(System Call)을 제공하며, 동일한 보안 행위라도 운영체제 버전이나 라이브러리 구현 방식에 따라 서로 다른 시스템 호출이 사용될 수 있다. 실제 엔터프라이즈급 EDR에서는 하나의 행위를 탐지하기 위해 여러 시스템 호출을 동시에 수집하여 상관 분석을 수행한다.
그러나 구현 범위와 성능을 고려하여 모든 시스템 호출을 수집하지 않고, 프로세스 생성, 네트워크 연결, 파일 생성, 파일 삭제, 프로세스 접근, 권한 상승 등 보안 행위를 대표할 수 있는 핵심 시스템 호출만을 선정하여 Audit Rule을 구성하였다.

[5-1-1] Auditd 설정 파일

(1) Auditd 서비스 설정

Auditd 서비스의 동작 방식은 다음 설정 파일에서 관리된다.

`/etc/audit/auditd.conf`

해당 파일에서는 다음과 같은 항목을 설정할 수 있다.

- 로그 저장 위치
 - 로그 파일 크기
 - 로그 순환 정책
 - 디스크 공간 부족 시 동작 방식
 - 버퍼 크기 및 성능 관련 옵션

(2) Auditd 규칙 설정

감사 규칙(Auditd Rule)은 다음 경로에 저장된다.

`/etc/auditd/rules.d/`

규칙은 하나의 파일에 작성할 수도 있으며, 목적에 따라 여러 파일로 분리하여 관리할 수도 있다.

예시)

```
/etc/audit/rules.d/audit.rules  
/etc/audit/rules.d/shadow.rules  
/etc/audit/rules.d/filewatch.rules
```

[5-1-2] Auditd 규칙의 구성 및 활용

Auditd는 감사 규칙(Audit Rule)을 통해 어떤 시스템 활동을 모니터링하고 기록할 것인지를 결정한다. 감사 규칙을 적절히 구성하면 Linux 커널에서 발생하는 이벤트 중 필요한 정보만 선택적으로 수집할 수 있으며, 불필요한 로그 발생을 줄여 시스템 성능 저하를 최소화할 수 있다.

Auditd 규칙은 크게 제어 규칙, 파일 시스템 규칙, 시스템 호출 규칙의 세 가지 유형으로 구분된다.

(1) 제어 규칙

제어 규칙은 특정 이벤트를 감시하기 위한 규칙이 아니라 Auditd 자체의 동작 방식을 설정하기 위한 규칙이다. 대표적인 제어 규칙은 다음과 같다.

```
-D  
-b 8192  
-f 1  
--backlog_wait_time 60000
```

각 설정의 의미는 다음과 같다.

규칙	설명
-D	기존 감사 규칙 삭제
-b 8192	커널 감사 버퍼 크기 설정
-f 1	감사 실패 시 Syslog 기록
--backlog_wait_time 60000	백로그 초과 시 대기 시간(ms)

(2) 파일 시스템 규칙

파일 시스템 규칙은 특정 파일 또는 디렉터리에 대한 접근을 감시하기 위해 사용된다.

```
-w [파일 경로] -p [권한] -k [키 이름]
```

각 옵션의 의미는 다음과 같다.

옵션	설명
-w	감시 대상 파일 또는 디렉터리
-p	접근 권한(r, w, a, x)
-k	이벤트 식별용 키

예를 들어, Linux 계정 정보가 저장된 /etc/shadow 파일을 감시하는 규칙은 다음과 같다.

```
-w /etc/shadow -p rwa -k shadow_access
```

/etc/shadow 파일에 대한 읽기, 쓰기, 속성 변경 행위를 기록하며, 생성된 이벤트를 shadow_access라는 키로 저장한다.

(3) 시스템 호출 규칙

시스템 호출 규칙은 Auditd의 핵심 기능으로, Linux 커널에서 발생하는 특정 시스템 호출을 직접 감시할 수 있다.

기본 형식은 다음과 같다.

```
-a [action],[filter] -S [syscall] -F [field=value] -k [key]
```

주요 옵션은 다음과 같다.

옵션	설명
-a	이벤트 기록 방식(규칙 추가)
-S	감시할 시스템 호출(Syscall) 지정
-F	추가 필터 조건
-k	이벤트 식별용 키(검색용 키)

EDR에서 자주 활용되는 시스템 호출은 다음과 같다.

시스템 호출	목적
execve	프로세스 실행
setuid	권한 상승
ptrace	프로세스 접근
connect	네트워크 연결
unlink	파일 삭제

예를 들어, 프로세스 실행을 감시하는 규칙은 다음과 같다.

```
-a always,exit -S execve -k process_create
```

해당 규칙은 프로세스 생성 시 발생하는 execve 시스템 호출을 기록하여 이후 탐지 엔진에서 분석할 수 있도록 한다.

[5-1-3] Audit Rule 적용 예시

```
# 1. Process Create
-a always,exit -F arch=b64 -S execve -S execveat -k process_create

# 2. Network Connect
-a always,exit -F arch=b64 -S connect -k network_connect

# 3. File Create
-a always,exit -F arch=b64 -S creat -k file_create

# 4. File Delete
-a always,exit -F arch=b64 -S unlink -S unlinkat -k file_delete

# 5. Process Access
-a always,exit -F arch=b64 -S ptrace -k process_access

# 6. Sensitive File Access
-w /etc/shadow -p rwa -k shadow_access
-w /etc/passwd -p rwa -k passwd_access

# 7. Privilege Escalation
-a always,exit -F arch=b64 -S setuid -S setgid -k privilege_escalation

# 8. Persistence
-w /etc/crontab -p wa -k persistence
-w /etc/systemd/system/ -p wa -k persistence

# SSH Authorized Keys
-w /root/.ssh/authorized_keys -p wa -k persistence
-w /home/pumpkinbee/.ssh/authorized_keys -p wa -k persistence

# Udev Rules
-w /etc/udev/rules.d/ -p wa -k persistence
-w /lib/udev/rules.d/ -p wa -k persistence
-w /usr/lib/udev/rules.d/ -p wa -k persistence
```

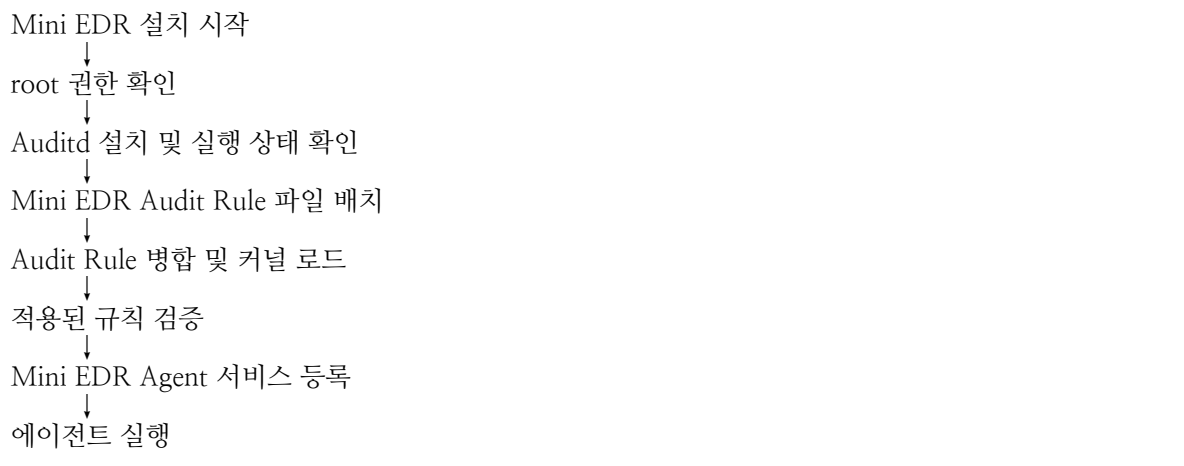

[5-1-4] Mini EDR 초기 설치 과정에서 Audit Rule 자동 설정

Mini EDR은 Linux Audit Framework에서 생성되는 보안 이벤트를 기반으로 동작하므로, 에이전트를 처음 설치할 때 수집 대상 시스템 콜과 파일 경로에 대한 Audit Rule 설정이 함께 이루어져야 한다.

사용자가 /etc/audit/rules.d/ 디렉터리의 설정 파일을 직접 생성하거나 수정하도록 요구하지 않고, Mini EDR 설치 프로그램이 초기 설정 과정에서 필요한 Audit Rule을 자동으로 설치하고 커널에 적용하도록 구성한다.

Audit Rule 설치하는 시스템 보안 설정을 변경하는 작업이므로 root 권한이 필요하다.

따라서 사용자는 Mini EDR 설치 프로그램을 sudo 또는 root 권한으로 한 번 실행하며, 설치 프로그램 내부에서는 다음 작업을 자동으로 수행한다.



프로젝트 저장소에는 Mini EDR 전용 Audit Rule 원본과 초기 설정 스크립트를 포함한다.



각 파일의 역할은 다음과 같다.

- project.rules
 - Mini EDR에서 수집할 시스템 콜과 파일 경로 감시 규칙을 정의한다.
- install.sh
 - Auditd 설치 여부와 실행 상태를 확인한다.
 - project.rules를 /etc/audit/rules.d/project.rules에 복사한다.
 - 규칙을 커널 Audit Subsystem에 로드한다.
 - Mini EDR Agent를 systemd 서비스로 등록하고 실행한다.
- uninstall.sh
 - Mini EDR Agent 서비스를 중지하고 제거한다.
 - 설치된 Audit Rule 파일을 제거한다.
 - 남아 있는 Audit Rule을 다시 커널에 적용한다.

Mini EDR 전용 Audit Rule은 시스템의 전역 Audit 설정과 분리하여 다음 위치에 설치한다.

```
/etc/audit/rules.d/audit.rules
```

기존 /etc/audit/rules.d/audit.rules는 Auditd의 전역 제어 설정을 담당하므로 Mini EDR 설치 프로그램에서 직접 덮어쓰지 않는다. Mini EDR은 전용 파일인 project.rules만 추가하여 기존 시스템 설정에 미치는 영향을 최소화한다.

설치 프로그램은 Rule 파일을 배치한 후 다음 명령을 내부적으로 실행한다.

```
augenrules -- load
```

augenrules는 /etc/audit/rules.d/ 아래의 .rules 파일들을 취합하여 최종 Audit Rule을 생성하고 Linux Kernel Audit Subsystem에 로드한다.

이후 다음 명령을 이용하여 Mini EDR에 필요한 규칙이 실제 커널에 등록되었는지 검증한다.

```
auditctl -l
```

설정 파일에 규칙이 존재하더라도 커널에 정상적으로 로드되지 않으면 Audit Event가 생성되지 않는다. 따라서 초기 설치 과정에서는 다음 세 단계를 모두 확인해야 한다.

```
/etc/audit/rules.d/project.rules 생성
↓
/etc/audit/audit.rules에 규칙 병합
↓
auditctl -l에서 커널 등록 여부 확인
```

설치 중 규칙 적용에 실패하면 Mini EDR Agent는 실행하지 않고 설치 오류를 사용자에게 출력한다. 이를 통해 수집 규칙이 없는 상태에서 에이전트만 실행되는 불완전한 설치를 방지한다.

최종적으로 사용자는 다음과 같이 Mini EDR 설치 프로그램만 실행하면 된다.

```
sudo ./scripts/install.sh
```

설치 프로그램이 Audit 규칙 설정, 규칙 적용 확인, Agent 서비스 등록 및 실행을 모두 자동으로 수행하므로 사용자가 Audit Rule 파일을 직접 수정할 필요는 없다.

install.sh

```
#!/usr/bin/env bash

set -euo pipefail

# 현재 실행 위치가 아니라 install.sh가 존재하는 디렉터리를 기준으로 한다.
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

RULE_FILE="${SCRIPT_DIR}/project.rules"
TARGET="/etc/audit/rules.d/project.rules"

# Audit Rule 설치에는 root 권한이 필요하다.
if [[ "${EUID}" -ne 0 ]]; then
    echo "[ERROR] root 권한으로 실행해야 합니다."
    echo "사용법: sudo ./install.sh"
    exit 1
fi

# 프로젝트 Audit Rule 원본 파일 존재 여부 확인
if [[ ! -f "${RULE_FILE}" ]]; then
    echo "[ERROR] Audit Rule 파일을 찾을 수 없습니다."
    echo "경로: ${RULE_FILE}"
    exit 1
fi

# auditd 명령 존재 여부 확인
if ! command -v augenrules >/dev/null 2>&1; then
    echo "[ERROR] augenrules 명령을 찾을 수 없습니다."
    echo "auditd가 설치되어 있는지 확인해야 합니다."
    exit 1
fi

echo "[INFO] Mini EDR Audit Rule을 설치합니다."

# root 소유권과 적절한 권한으로 규칙 파일 설치
install W
    -o root W
    -g root W
    -m 0640 W
    "${RULE_FILE}" W
    "${TARGET}"

echo "[INFO] Audit Rule을 커널에 적용합니다."

augenrules --load

echo "[INFO] 적용된 Audit Rule을 확인합니다."

if ! auditctl -l | grep -Eq W
    'process_create|network_connect|file_create|file_delete|process_access|shadow_access|passwd_access|
    privilege_escalation|persistence'; then
    echo "[ERROR] Mini EDR Audit Rule이 커널에 정상적으로 등록되지 않았습니다."
    exit 1
fi

echo "[SUCCESS] Mini EDR Audit Rule 설치가 완료되었습니다."
```

uninstall.sh

```
#!/usr/bin/env bash

set -euo pipefail

TARGET="/etc/audit/rules.d/project.rules"

# Audit Rule 제거에는 root 권한이 필요하다.
if [[ "${EUID}" -ne 0 ]]; then
    echo "[ERROR] root 권한으로 실행해야 합니다."
    echo "사용법: sudo ./uninstall.sh"
    exit 1
fi

if [[ -f "${TARGET}" ]]; then
    echo "[INFO] Mini EDR Audit Rule을 제거합니다."
    rm -f "${TARGET}"
else
    echo "[INFO] 설치된 Mini EDR Audit Rule 파일이 없습니다."
fi

echo "[INFO] 남아 있는 Audit Rule을 다시 커널에 적용합니다."

augenrules --load

echo "[INFO] Mini EDR Audit Rule 제거 여부를 확인합니다."

if auditctl -l | grep -Eq W
'process_create|network_connect|file_create|file_delete|process_access|shadow_access|passwd_access|
privilege_escalation'; then
    echo "[WARNING] 일부 Mini EDR Audit Rule이 커널에 남아 있습니다."
    echo "sudo auditctl -l 명령으로 확인해야 합니다."
    exit 1
fi

echo "[SUCCESS] Mini EDR Audit Rule 제거가 완료되었습니다."
```

실행 권한 설정

```
chmod +x scripts/install.sh
chmod +x scripts/uninstall.sh
```

설치:

```
sudo ./scripts/install.sh
```

제거:

```
sudo ./scripts/uninstall.sh
```

[5-2] User Space – Collector

[5-2-1] Collector의 역할

Collector는 Linux Kernel의 Audit Subsystem을 통해 생성된 Audit 로그를 사용자 공간(User Space)에서 수집하는 첫 번째 컴포넌트이다.

커널은 Audit Rule과 일치하는 이벤트가 발생하면 해당 이벤트를 /var/log/audit/audit.log 파일에 기록한다. Collector는 이 로그 파일을 실시간으로 모니터링(Tailing)하여 새롭게 생성되는 Audit Record를 수집하고, 동일한 Audit Event ID를 기준으로 여러 Record를 하나의 이벤트로 통합(Event Aggregation)한다. 이후 Event Dispatcher에서 활용할 수 있도록 AuditLogGroup 객체를 생성하여 전달한다.

Collector의 주요 기능은 다음과 같다.

- Audit 로그 실시간 수집
- Audit Record 파싱
- 동일 Audit ID 기반 이벤트 조립(Event Aggregation)
- SystemEvent 생성
- Event Dispatcher로 이벤트 전달

[5-2-2] Collector 구현

Collector는 Audit 로그를 실시간으로 수집하여 AuditLogGroup을 생성하기까지 여러 단계를 거쳐 구현하였다. 구현 과정은 Audit 로그 실시간 수집, Audit Record 매핑, Event Aggregation, 테스트, 예외 처리, 동시성 개선의 순서로 진행하였다.

(1) Audit 로그 실시간 수집

먼저 Audit 로그를 지속적으로 모니터링하기 위해 tail.go를 구현하였다.

TailEngine은 /var/log/audit/audit.log 파일을 실시간으로 감시(Tailing)하여 새롭게 생성되는 Audit Record를 읽어온다. 프로그램 실행 이전의 로그는 읽지 않고 파일의 마지막 위치부터 모니터링하도록 구성하여, 에이전트 실행 이후 발생하는 이벤트만 수집하도록 구현하였다.

수집된 Audit Record는 Channel을 통해 AuditCollector로 전달되며, 이를 통해 로그 수집과 이벤트 조립을 서로 독립적으로 수행할 수 있도록 구성하였다.

(2) Audit Record 매핑

수집된 Audit 로그는 문자열 형태이므로 직접 활용하기 어렵다.

따라서 Record Type을 판별한 후 필요한 Field를 추출하여 대응되는 Go 구조체로 변환하는 과정을 구현하였다.

현재 Collector에서 처리하는 주요 Audit Record는 다음과 같다.

- SYSCALL
- EXECVE
- PATH
- CWD

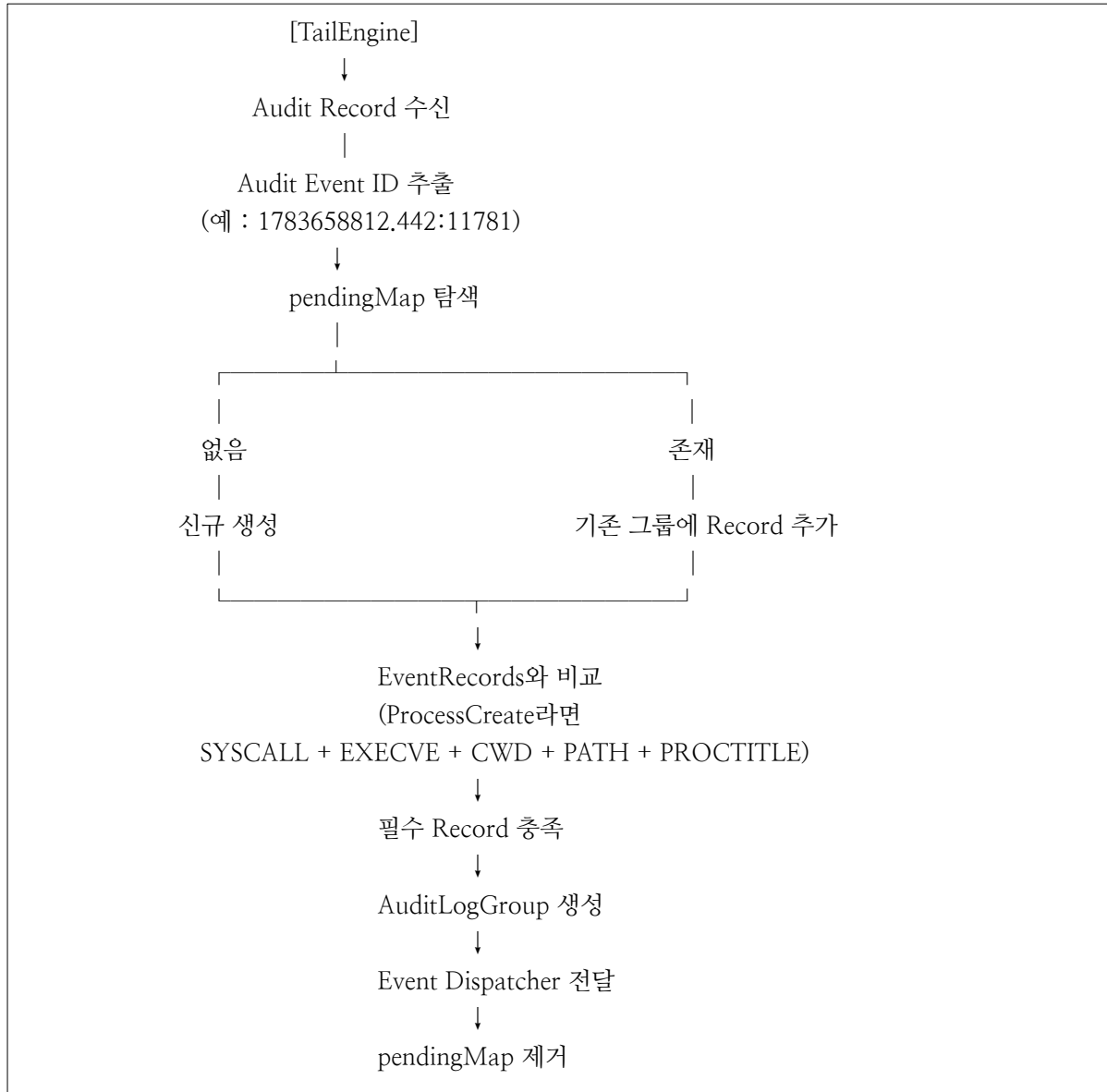
- PROCTITLE
- SOCKADDR

각 Record는 Record Type에 맞는 구조체로 매핑되며, 이후 Event Aggregation 과정에서 하나의 이벤트를 구성하는 데이터로 활용된다.

(3) Event Aggregation 및 SystemEvent 생성

Auditd는 하나의 보안 이벤트를 여러 개의 Audit Record로 분리하여 기록한다.

Collector는 동일한 Audit Event ID(serial)를 기준으로 관련 Record를 하나의 이벤트로 통합하도록 구현한다.



Event Type마다 필요한 Record는 types.go의 EventRecords에 정의되어 있으며, Collector는 현재까지 수집된 Record와 요구되는 Record를 비교하여 이벤트의 완성 여부를 판단한다.

필수 Record가 모두 수집되면 하나의 AuditLogGroup을 생성하여 Event Dispatcher로 전달하고, 해당 Event ID는 pendingMap에서 즉시 제거하여 메모리 사용량을 최소화하였다.

[5-3] User Space – Event Dispatcher

Event Dispatcher는 Collector에서 Event Aggregation이 완료된 Audit Event를 Rule Engine에서 사용할 수 있는 표준 이벤트 객체(SystemEvent)로 변환하는 역할을 수행한다.

Collector에서는 Audit Record를 수집하고 하나의 Event로 조립하는 작업까지만 수행하며, Dispatcher는 이러한 Event를 Rule Engine이 사용할 수 있는 공통 데이터 모델로 변환한다. 이를 통해 Rule Engine은 Audit 로그의 복잡한 구조를 알 필요 없이 동일한 형식의 이벤트만을 처리할 수 있다.

Event Dispatcher의 구현 과정은 SystemEvent 설계, Event 변환, 부가 정보 생성, 테스트, 예외 처리의 순서로 진행된다.

(1) SystemEvent 데이터 모델 설계

Collector에서 생성된 Event는 Audit Record 구조를 그대로 유지하고 있으므로 Rule Engine에서 직접 사용하기에는 적합하지 않다.

따라서 Rule Engine이 필요한 정보만 사용할 수 있도록 공통 이벤트 모델(SystemEvent)을 설계한다.

SystemEvent에는 이벤트 종류(EventType), 프로세스 정보, 부모 프로세스, 명령행, 파일 경로, 네트워크 정보, 사용자 정보 등 Rule Engine에서 공통적으로 사용하는 필드만 포함한다.

이를 통해 Rule Engine은 ProcessCreate, NetworkConnect, FileCreate 등 서로 다른 이벤트를 동일한 인터페이스로 처리할 수 있다.

(2) Audit Event → SystemEvent 변환

Dispatcher는 Collector에서 전달받은 Event를 분석하여 SystemEvent를 생성한다.

EventType에 따라 필요한 Audit Record(SYS CALL, EXECVE, PATH, SOCKADDR 등)의 값을 추출하고, 이를 SystemEvent의 각 필드에 매핑한다.

예를 들어,

- ProcessCreate → 실행 파일, 부모 프로세스, Command Line 생성
- FileCreate/FileDelete → 대상 파일 경로 생성
- NetWorkConnect → 목적지 IP와 Port 생성

과 같이 EventType별 필요한 정보를 하나의 표준 이벤트 객체로 변환한다.

(3) 추가 정보 생성

Audit 로그에는 직접 기록되지 않는 정보도 존재한다.

따라서 Dispatcher에서는 Rule Engine이 활용하기 쉬운 형태로 일부 정보를 추가 생성한다. 대표적으로 다음과 같은 정보가 생성된다.

- 부모 프로세스 실행 파일(Parent Image)
- Process Name
- Command Line
- Network Protocol
- Target File

예를 들어, Parent PID만 존재하는 경우에는 /proc/<PPID>/exe를 조회하여 Parent Image를 생성한다. 이러한 정보는 Rule Engine에서 부모 프로세스 기반 탐지나 행위 기반 탐지를 수행하기 위해 사용된다.

[5-4] User Space – Rule Engine

Rule Engine은 Event Dispatcher에서 생성한 SystemEvent를 입력받아 미리 정의된 탐지 규칙(rule)과 비교하여 악의적인 행위를 탐지하는 핵심 모듈이다. Rule Engine은 JSON 형식으로 작성된 규칙 파일을 프로그램 시작 시 메모리에 적재하며, 이벤트가 발생할 때마다 해당 이벤트와 모든 규칙을 비교하여 탐지 여부를 판단한다.

먼저 Rule Engine은 이벤트의 EventType과 규칙의 EventType을 비교하여 관련 없는 규칙을 즉시 제외한다. 이를 통해 하나의 이벤트에 대해 모든 규칙을 검사하지 않고 동일한 이벤트 유형의 규칙만 비교하도록 하여 불필요한 연산을 최소화하였다.

이후 조건 비교 단계에서는 Rule에 정의된 각 Condition을 순차적으로 검사한다. Rule Engine은 Dispatcher에서 생성한 SystemEvent로부터 필요한 필드 값을 추출한 뒤 RuleMatcher를 이용하여 규칙 조건과 비교한다. 문자열 비교, 숫자 비교, 그룹(Group)비교, 와일드카드(Path Pattern) 비교 등을 지원하도록 구현하였으며, 모든 조건이 만족되는 경우에만 해당 Rule를 탐지 결과로 판단한다.

또한 문자열 비교 과정에서는 명령행(CommandLine)을 단순 문자열 포함 여부로 비교하지 않고 토큰(Token) 단위로 분리하여 검사하도록 개선하였다. 이를 통해 ls -color=auto와 같이 정상 명령어 sh, id 등의 짧은 문자열과 우연히 일치하여 탐지되는 오탐을 줄이고, 실제 명령어 단위의 정확한 비교가 가능하도록 구현하였다.

탐지가 완료되면 Rule Engine은 탐지된 Rule ID와 함께 Alert 이벤트를 생성하여 Alert Dispatcher로 전달한다. Alert에는 탐지 규칙 정보뿐만 아니라 Process 정보, CommandLine, TargetFile, Network 정보 등 분석에 필요한 상세 정보를 함께 포함하여 이후 대응 및 저장 단계에서 활용할 수 있도록 구성하였다.

또한 Rule Engine은 RuleMatcher를 별도의 모듈로 분리하여 구현하였다.

이를 통해 Rule Engine은 이벤트 흐름과 탐지 절차만 담당하고, 문자열 비교, 그룹 비교, 와일드카드 비교 등의 세부 비교 알고리즘은 RuleMatcher가 담당하도록 역할을 분리하였다. 이러한 구조는 새로운 비교 방식이 추가되더라도 Rule Engine의 수정 없이 RuleMatcher만 확장하면 되므로 유지보수성과 확장성을 높일 수 있다.

[5-4] User Space - Alert

(1) Alert의 역할

Alert는 Rule Engine에서 탐지된 보안 이벤트를 사용자에게 전달하고, 이후 저장 및 대응 기능으로 연계하기 위한 사용자 공간의 마지막 처리 계층이다.

Rule Engine은 수집된 SystemEvent와 탐지 규칙(Rule)을 비교하여 이벤트의 탐지 여부만을 판단한다. 그러나 탐지 결과를 출력하거나 저장하고, 이후 대응 기능까지 Rule Engine이 직접 수행할 경우 하나의 컴포넌트가 과도한 책임을 가지게 되어 유지보수가 어려워진다.

따라서 탐지 기능과 후처리 기능을 분리하기 위해 Alert 계층을 독립적으로 설계하였다.

Alert는 Rule Engine으로부터 탐지 결과를 전달받아 사용자에게 탐지 정보를 제공하고, 향후 저장 모듈과 대응 모듈로 전달할 수 있는 구조를 제공한다.

Alert 계층의 주요 역할은 다음과 같다.

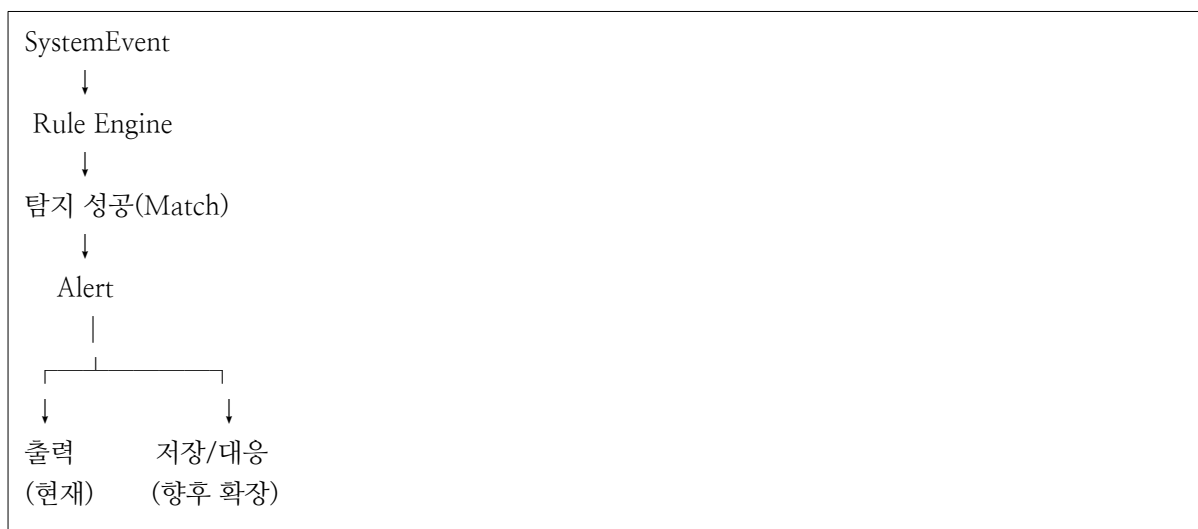
- Rule Engine의 탐지 결과 수신
- 탐지 결과 출력
- 저장(Storage) 기능으로 전달
- 대응(Response) 기능으로 전달

이를 통해 Rule Engine은 탐지 기능에만 집중하고, Alert는 탐지 이후의 후처리를 담당하도록 역할을 분리하였다.

(2) Alert 처리 흐름

Rule Engine에서 탐지 조건을 모두 만족한 이벤트는 Alert 계층으로 전달된다. Alert는 전달받은 탐지 결과를 사용자에게 출력하고, 이후 저장 및 대응 기능으로 전달할 수 있도록 설계하였다.

전체 처리 과정은 다음과 같다.



현재 프로젝트에서는 Alert의 기본 기능으로 탐지 결과를 콘솔에 출력하도록 구현하였다.

향후에는 동일한 구조를 활용하여 JSON 파일 저장, 데이터베이스 저장, Dashboard 저장, 자동 대응 등의 기능을 추가할 수 있도록 확장성을 고려하였다.

(3) Alert 계층을 분리한 이유

탐지 기능과 후처리 기능을 하나의 컴포넌트에서 모두 수행할 경우 시스템의 결합도가 증가하고 유지보수가 어려워진다.

예를 들어, Rule Engine이 탐지뿐만 아니라 탐지 결과 출력, 파일 저장, 데이터베이스 저장, 네트워크 전송, 프로세스 종료 등의 기능을 모두 수행하도록 구현하면 새로운 기능이 추가될 때마다 Rule Engine을 수정해야 한다.

반면 Alert 계층을 별도로 구성하면 Rule Engine은 탐지 기능만 담당하고, Alert는 탐지 이후의 처리만 담당하게 된다.

이와 같은 계층 분리를 통해 각 컴포넌트의 책임을 명확하게 구분할 수 있으며, 향후 저장 기능이나, 자동 대응 기능이 추가되더라도 기존 탐지 로직을 수정하지 않고 기능을 확장할 수 있다.

[7] Mini EDR Rule

[7-1] EDR Rule Engine

[7-1-1] 개요

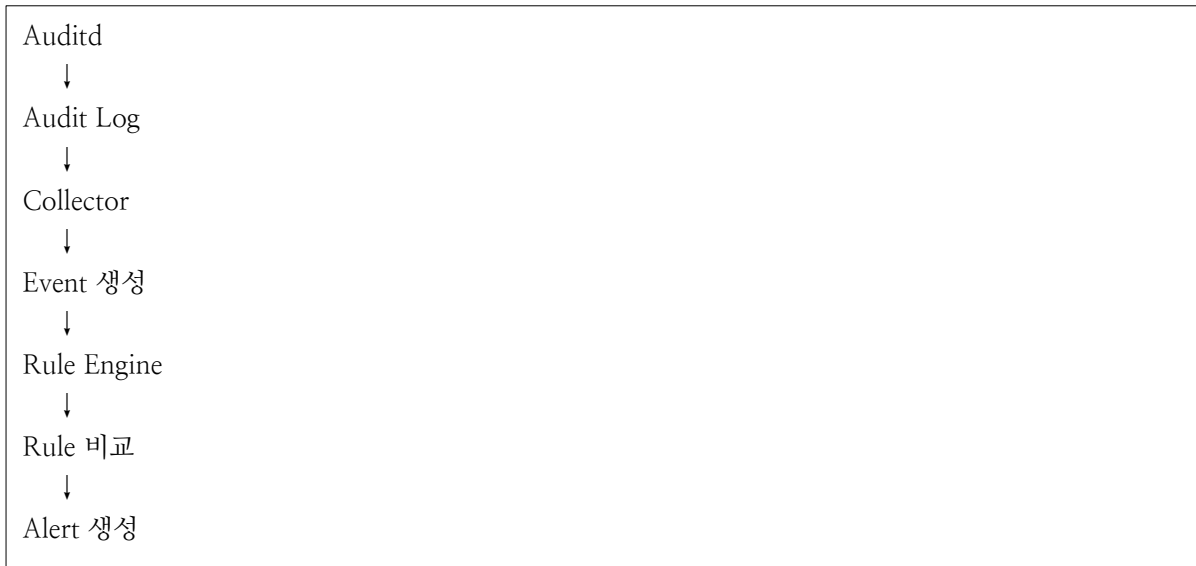
EDR의 Rule Engine은 엔드포인트에서 수집된 다양한 이벤트를 분석하여 잠재적인 위협을 탐지하는 핵심 구성 요소이다. 운영체제에서는 프로세스 생성, 파일 접근, 네트워크 연결, 권한 변경 등 다양한 이벤트가 지속적으로 발생한다. 그러나 이러한 이벤트는 정상적인 시스템 동작에서도 빈번하게 발생하기 때문에 단순히 이벤트의 발생 여부만으로는 악성 여부를 판단할 수 없다.

따라서 Rule Engine은 이벤트의 발생 여부뿐만 아니라 실행 파일, 명령행 인자, 접근 대상, 사용자 정보 등 이벤트의 속성을 함께 분석하여 미리 정의된 탐지 규칙과 비교한다.

조건을 만족하는 경우에는 해당 이벤트를 위협으로 판단하고 경고(Alert)를 생성하거나 대응(Response)을 수행한다.

[7-1-2] Rule Engine의 동작 과정

Rule Engine은 Auditd에서 생성된 로그를 기반으로 생성된 Event를 입력받아 탐지 규칙과 비교한 후 탐지 여부를 판단한다.



이 과정에서 Rule Engine은 단순히 Event의 종류만 비교하는 것이 아니라, Event 내부의 다양한 속성을 함께 분석한다.

[7-1-3] Rule의 구성 요소

하나의 Rule은 일반적으로 탐지 대상(Event)과 탐지 조건(Condition)으로 구성된다.

Event는 Rule이 적용될 대상이며, Condition은 Event가 만족해야 하는 조건이다.

예를 들어, ProcessCreate Event에서는 다음과 같은 속성을 활용할 수 있다.

속성	설명
exe	실행 파일 경로
command	실행 프로그램
pid	프로세스 ID
ppid	부모 프로세스 ID

uid	실행 사용자
arguments	명령행 인자
time	실행 시간

이러한 속성을 조합하여 Rule을 구성한다.

예를 들어, ProcessCreate Event에서 exe=/usr/bin/bash인 경우에는 셸 실행으로 판단할 수 있으며, NetworkConnect Event에서 목적지 IP가 내부망이 아닌 경우에는 외부 네트워크 연결로 판단할 수 있다.

[7-1-4] Rule의 탐지 방식

Rule Engine은 크게 단일 이벤트 기반 탐지(Event-based Detection)와 상관관계 기반 탐지(Correlation Detection)로 구분할 수 있다.

구분	설명
Event-based Detection	하나의 Event만 분석하여 탐지
Correlation Detection	여러 Event를 시간 순서대로 분석하여 탐지

단일 이벤트 기반 탐지는 하나의 이벤트만으로 공격 여부를 판단하는 방식이다.

예를 들어, Netcat 실행이나 /etc/shadow 접근과 같이 단일 이벤트만으로도 의미 있는 탐지가 가능한 경우에 사용된다.

반면 상관관계 기반 탐지는 여러 이벤트를 시간 순서대로 분석하여 공격 패턴을 식별한다.

예를 들어, 외부에서 파일을 다운로드한 후 즉시 실행하는 경우 각각의 Event는 정상적으로 보일 수 있지만, 연속적인 이벤트 흐름을 분석하면 악성 코드 실행이라는 공격 시나리오를 식별할 수 있다.

[7-1-5] MITRE ATT & CK 기반 Rule 설계

현대 EDR 제품은 대부분 MITRE ATT&CK Framework를 기반으로 탐지 규칙을 설계한다.

MITRE ATT&CK는 운영체제의 System Call을 설명하는 데이터베이스가 아니라 공격자가 수행하는 행위(TTP)를 정의한 Framework이다. 따라서 Rule은 System Call 자체를 탐지하는 것이 아니라, System Call을 통해 생성된 Event가 어떤 공격 행위를 의미하는지를 분석하여 MITRE ATT&CK Technique와 연결한다.

예를 들어, execve()는 프로그램을 실행하기 위한 System Call이지만,

공격자는 Bash 실행, Python 실행, 악성코드 실행 등 다양한 공격 과정에서 이를 사용한다.

따라서 Rule은 execve() 자체를 탐지하는 것이 아니라 ProcessCreate Event를 기반으로 Command Execution 여부를 판단하도록 설계한다.

[7-1-6] 참고 자료

https://edr.datto.com/help/Content/04-configuring-assigning-policies/general-policy/what-are-detection-rules.htm?utm_source=chatgpt.com

[7-2] Mini EDR Rule 설계 절차 설명

[7-2-1] Rule 설계 방향

Mini EDR의 Rule Engine은 Linux Audtid에서 수집한 Audit Event를 기반으로 공격자의 행위를 탐지하는 것을 목적으로 한다. Rule을 설계하기 위해서는 단순히 System Call을 분석하는 것이 아니라, Audit Event가 의미하는 행위를 MITRE ATT&CK Technique와 연결하고, 각 Technique에서 제시하는 Detection Strategy를 참고하여 탐지 규칙을 도출하는 과정이 필요하다.

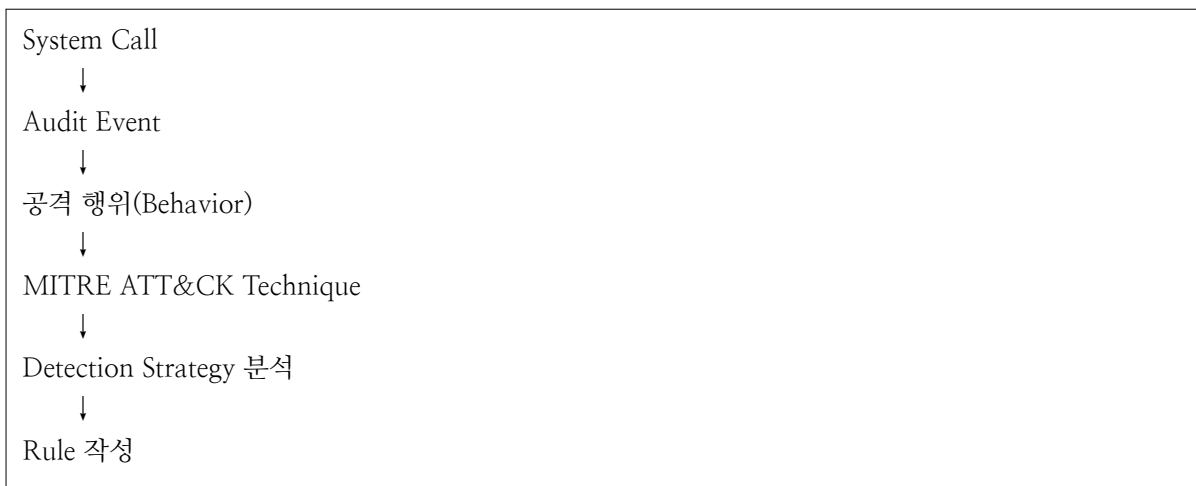
여기서 주의할 점은 System Call 자체를 공격으로 판단하지 않는다는 것이다.

execve(), connect(), unlink()와 같은 System Call 운영체제가 제공하는 기본 기능이며, 정상적인 프로그램도 동일한 System Call을 사용한다. 따라서 특정 System Call이 발생했다는 사실만으로는 공격 여부를 판단할 수 없다.

따라서 Rule 설계는 “어떤 System Call이 발생했는가”보다는 “해당 System Call이 어떤 행위를 의미하며, 공격자는 이러한 행위를 어떤 방식으로 악용하는가”를 중심으로 이루어진다.

이후 해당 행위와 관련된 MITRE ATT&CK Technique 및 Detection Strategy를 분석하여 실제 탐지 가능한 Rule을 작성한다.

이를 다음과 같은 흐름으로 정리할 수 있다.



[7-2-2] Audit Event 정의

Rule 설계의 첫 번째 단계는 Linux Audtid에서 수집할 Event를 정의하는 것이다.

예를 들어, execve()는 프로그램 실행, connect()는 네트워크 연결, unlink()는 파일 삭제와 같은 행위를 나타낸다.

이 단계에서는 공격 여부를 판단하지 않고, 운영체제에서 어떤 행위가 발생했는지를 Event로 정의하는 과정이다. 예를 들어 다음과 같이 Event를 분류할 수 있다.

Audit Event	Auditd 규칙 예시
ProcessCreate	execve, execveat
NetworkConnect	connect
FileCreate	creat

[7-2-3] MITRE ATT&CK Technique 및 Detection Strategy 분석

Audit Event가 정의되면 각 Event가 어떤 공격 기법에서 사용되는지를 MITRE ATT&CK를 통해 분석한다.

여기서 중요한 점은 System Call 자체를 검색하는 것이 아니라, System Call이 의미하는 행위를 기준으로 Technique를 탐색하는 것이다.

예를 들어, `execve()`는 프로그램을 실행하기 위한 System Call이다. 따라서 ATT&CK에서 `execve()`를 검색하는 것이 아니라, Shell 실행, 스크립트 실행, 프로세스 실행과 같은 행위를 기준으로 관련 Technique를 찾는다.

동일한 방식으로 `connect()`는 외부와의 네트워크 연결을 의미하므로 Command and Control(C2), Application Layer Protocol 등의 Technique와 연결할 수 있다.

Technique가 걸렸다면 해당 Technique에서 제공하는 Detection Strategy를 분석한다. Detection Strategy는 “이러한 조건을 탐지하라”는 탐지 방향을 설명하는 문서이며, 실제 Rule을 직접 제공하는 것은 아니다.

예를 들어, ATT&CK의 T1059.004(Unix Shell)에서는 다음과 같이 설명한다.

Detects bash, sh, zsh, or BusyBox shell execution initiated via remote sessions, unauthorized users...

이 문장은 bash를 탐지하는 Rule을 작성하라는 의미가 아니라, bash, sh, zsh BusyBox가 특정 상황에서 실행되는 행위를 탐지해야 한다는 의미이다.

따라서 Detection Strategy를 Linux Auditd에서 확인 가능한 정보로 변환하는 과정이 필요하다.

예를 들어, Auditd에서는 실행된 프로그램의 경로(exe), 부모 프로세스(ppid), 실행 인자(argv) 등의 정보를 확인할 수 있으므로 Detection Strategy를 다음과 같은 Rule 후보로 변환할 수 있다.

Detection Strategy	Auditd 정보	Rule 예시
bash 실행	exe	exe == "/bin/bash"
sh 실행	exe	exe == "/bin/sh"
BusyBox 실행	exe	exe == "/bin/busybox"
Remote Session	부모 프로세스	parent == "sshd"
Suspicious Command	argv	argv contains "curl"
Network Discovery	argv	argv contains "ifconfig"

즉, Detection Strategy를 탐지해야 하는 행위를 설명하는 문서이며, Rule은 해당 행위를 Audit Event에서 어떤 필드로 판별할 것인지를 정의하는 것이다.

[7-2-4] 분석 범위 설정

MITRE ATT&CK는 수백 개의 Technique와 Sub-technique로 구성되어 있다.

모든 Technique를 대상으로 Rule를 설계하는 것은 현실적으로 어렵기 때문에 프로젝트 범위를 명확하게 설정할 필요가 있다.

선정된 Technique는 Auditd를 통해 수집 가능한 Event와 직접적으로 연관되며, Detection Strategy를 Audit Event로 변환하여 Rule을 작성할 수 있는 항목을 우선 대상으로 한다.

이를 통해 Rule과 검증을 체계적으로 수행할 수 있으며, 이후 새로운 Technique와 Detection Strategy를 추가하여 Rule을 점진적으로 확장할 수 있다.

예를 들어, Mini EDR에서는 대표적인 Linux 공격 행위를 중심으로 다음 Technique를 우선 대상으로 선정한다.

Technique	탐지 목적
T1059	Command Execution
T1071	C2 통신
T1070	로그 및 증거 삭제
T1055	Process Injection
T1003	Credential Dumping
T1068	Privilege Escalation
T1547	Persistence

범위를 제한하면 Rule 작성과 검증을 체계적으로 수행할 수 있으며, 이후 필요에 따라 Technique를 점진적으로 확장할 수 있다.

[7-3] Mini EDR Rule 설계

System Call	기능	생성되는 Event
execve	새로운 프로그램 실행	ProcessCreate
execveat	새로운 프로그램 실행(상대 경로 지원)	ProcessCreate
connect	네트워크 연결 요청	NetworkConnect
creat	파일 생성	FileCreate
unlink	파일 삭제	FileDelete
unlinkat	파일 삭제	FileDelete
ptrace	다른 프로세스 접근	ProcessAccess
setuid	UID 변경	PrivilegeEscalation
setgid	GID 변경	PrivilegeEscalation

Audit Rule	EventType
process_create	ProcessCreate
network_connect	NetworkConnect
file_create	FileCreate
file_delete	FileDelete
process_access	ProcessAccess
shadow_access	ShadowAccess
passwd_access	PasswdAccess
privilege_escalation	PrivilegeEscalation
persistence	Persistence

[7-3-1] ProcessCreate Rule 설계

(1) System Call

ProcessCreate Event는 `execve()`와 `execveat()` System Call을 기반으로 생성된다.

① `execve()`란

현재 프로세스를 완전히 새로운 프로그램으로 교체하여 실행하는 핵심 시스템 콜이다. 기존 프로세스의 메모리, 스택, 힙을 모두 새 프로그램으로 덮어쓰우며, PID(프로세스 ID)는 유지된다.

```
int execve(const char *filename, char *const argv[], char *const envp[]);
```

- filename: 실행할 파일 경로
- argv: 프로그램에 전달할 인자 배열이며, 배열의 마지막은 반드시 NULL로 끝나야 한다.
- envp: 프로그램에 전달할 환경 변수 배열이며, 이 역시 마지막은 NULL로 끝나야 한다.

② `execveat()`란

지정된 디렉터리 파일 디스크립터(File Descriptor)를 기준으로 상대 경로에서 새로운 프로그램을 실행하는 리눅스 시스템 콜이다. 기본적인 역할은 `execve()`와 동일하지만, 경로 기준점을 프로세스의 현재 작업 디렉터리 대신 특정 디렉터리로 지정할 수 있다는 유연성이 있다.

이 시스템 콜을 사용하면 현재 디렉터리 경로에 영향을 받지 않고 안전하게 파일을 실행하거나 특정 디렉터리 권한을 활용해 프로그램을 제어할 수 있다.

```
int execveat(int dirfd, const char *pathname, char *const argv[], char *const envp[], int flags);
```

- dirfd: 기준이 될 디렉터리의 파일 디스크립터이다. `AT_FDCWD`를 넘기면 `execve()`처럼 현재 작업 디렉터리를 기준으로 경로를 해석한다.
- pathname: 실행할 파일의 경로이다.
- argv/envp: 실행할 프로그램의 인자(Argument)와 환경 변수이다.
- flags: 추가적인 동작 방식을 제어하는 비트마스크이다. 빈 문자열 경로와 `AT_EMPTY_PATH` 플래그를 결합하면 dirfd가 가리키는 파일 자체를 직접 실행할 수도 있다.

(2) MITRE ATT&CK – 탐지 전략 종류

① ProcessCreate(DC0032)

운영체제가 새로운 프로세스(실행 파일)를 초기화하는 이벤트를 의미한다.
이 과정에는 부모-자식 프로세스 관계, 프로세스 인자, 환경 변수 등이 포함될 수 있다.
프로세스 생성 과정을 모니터링하는 것은 승인되지 않은 바이너리 실행, 스크립팅 악용, 권한 상승 시도와 같은 악의적인 행위를 탐지하는 데 매우 중요하다.

② Detection Strategy

Web Shell 탐지

- ID: DET0394
- 이름: Web Shell Detection via Server Behavior and File Execution Chains
- MITRE ID: T1505.003(Server Software Component: Web Shell)
- 설명: Linux 서버 침해의 대다수는 Apache, Nginx, Tomcat 같은 웹 서비스의 취약점을 통해 시작된다. 웹 서버 프로세스(httpd, nginx, java 등)가 갑자기 시스템 명령행 인터페이스(sh, bash)를 자식 프로세스로 생성하여 실행하는 비정상적인 행위 체인을 탐지하는 가장 필수적인 전략이다.

Unix 셸 명령 수행 및 스크립트 오용 탐지

- ID: DET0384
- 이름: Behavioral Detection of Unix Shell Execution
- MITRE ID: T1059.004(Command and Scripting: Unix Shell)
- 설명: 공격자가 Linux 시스템에 침투한 후 정보를 수집하거나 악성 스크립트를 실행할 때 bash, sh, dash 등을 무조건 사용하게 된다. 일반적인 개발자나 관리자의 명령어 패턴(정상 배포 스크립트 등)과 다른, 외부 유입 경로나 임시 디렉터리(/tmp, /dev/shm)에서의 셸 실행을 잡아내는 대중적인 전략이다.

/proc 파일시스템 접근을 통한 자격 증명 탈취 탐지

- ID: DET0593
- 이름: Detecting OS Credential Dumping via /proc Filesystem Access on Linux
- MITRE ID: T1003.007(OS Credential Dumping: /proc Filesystem)
- 설명: Windows에 LSASS 덤핑이 있다면, Linux에는 /proc 메모리 접근이 있다. 공격자는 메모리 상에 남아있는 자격 증명(비밀번호, API 키 등)을 캐내기 위해 특정 프로세스의 /proc/[PID]/mem 이나 /proc/[PID]/maps 파일을 파싱하려고 시도한다. 이 비정상적인 파일 접근 패턴을 모니터링한다.

Linux 흔적 지우기(안티 포렌식) 탐지

- ID: DET0520
- 이름: Behavioral Detection of Log File Clearing on Linux and macOS
- MITRE ID: T1685.006(Indicator Removal: Log File Clearing)
- 설명: 공격자들은 자기가 들어온 흔적을 감추기 위해 Linux의 대표적인 로그 파일인 /var/log/auth.log, secure, wtmp 등을 변조하거나 지우려고 한다. 또는 history -c 명령어나 unset HISTFILE 등을 통해 이력을 숨기려 하는데, 이를 실시간으로 탐지하는 전략이다.

Cron Job 및 Systemd를 통한 지속성(Persistence) 확보 탐지

- ID: DET0290
- 이름: Cross-Platform Detection of Cron Job Abuse for Persistence and Execution
- MITRE ID: T1053.003(Scheduled Task/Job: Cron)
- 선정 이유: Linux 환경에서 공격자가 권한을 잃지 않고 계속 남아있기 위해 애용하는 방식이다. /etc/cron* 디렉토리나 /var/spool/cron/crontabs/ 내에 악성 예약 명령을 등록하거나, systemd 서비스 유닛 파일을 몰래 생성하여 재부팅 시에도 악성코드가 자동 실행되도록 만드는 행위를 감시한다.

(3) MITRE ATT&CK - 탐지할 선정 공격 방법

MITRE ATT&CK에서는 Process Create 이벤트를 활용한 다양한 Detection Strategy를 제공한다. 그러나 모든 탐지 전략을 구현하기에는 프로젝트의 범위와 개발 기간에 한계가 있으므로, 이 프로젝트에서는 Linux 환경에서 대표적으로는 활용되는 Unix 셸 명령 수행 및 스크립트 오용 탐지(DET0384)를 Reul Engine의 탐지 규칙으로 선정하였다.

T1059.004 - Command and Scripting Interpreter: Unix Shell

공격자는 Unix 셸을 이용하여 시스템에서 명령을 실행하거나 악성 스크립트를 수행할 수 있다. Unix 셸은 Linux, macOS, ESXi에서 사용되는 기본 명령어 실행 환경으로, 사용자가 입력한 명령을 운영체제에 전달하여 실행하는 역할을 한다. 대표적인 셸로는 sh, bash, zsh, ash 등이 있다.

셸에서는 단순히 명령어 하나만 실행하는 것이 아니라, 여러 명령을 하나의 셸 스크립트로 작성하여 순서대로 실행할 수도 있다. 이러한 기능은 원래 반복적인 작업을 자동화하기 위해 사용되지만, 공격자 역시 이를 악용할 수 있다.

예를 들어, 공격자는 셸을 이용해 다음과 같은 작업을 수행할 수 있다.

- 악성 파일 다운로드
- 다운로드한 파일 실행
- 시스템 정보 수집
- 관리자 권한 획득 시도
- 원격 서버와 연결하여 명령 실행
- 여러 명령을 하나의 스크립트로 자동 실행

또한 공격자는 SSH나 원격 제어(C2) 서버를 통해 시스템의 셸에 접근하여 직접 명령을 실행할 수도 있다. 또는 셸 스크립트를 이용해 여러 악성 명령을 한 번에 실행하거나, 시스템에 지속적으로 접근할 수 있도록 설정하기도 한다.

일부 임베디드 장치나 경량 Linux 시스템에서는 BusyBox가 사용된다. BusyBox는 여러 Linux 명령어와 간단한 셸 기능을 하나의 작은 프로그램으로 제공하는 도구이다. 공격자는 이러한 환경에서도 일반적인 Linux 시스템과 비슷한 방식으로 명령을 실행하거나 스크립트를 수행할 수 있다.

(4) Detection Strategy 분석

① AN1081

AN1081은 Unix Shell(bash, sh, dash, BusyBox 등)을 이용하여 수행되는 공격 행위를 탐지하기 위한 분석 기법이다. 공격자는 시스템 침해 이후 대부분의 명령을 Unix Shell을 통해 실행하며, 이를 이용하여 시스템 정보 수집, 파일 다운로드, 네트워크 탐색, Reverse Shell 생성, 지속성 확보 등의 후속 공격을 수행한다.

그러나 Linux 환경에서는 관리자나 일반 사용자 역시 Shell을 이용하여 정상적인 작업을 수행하므로, 단순히 Shell 실행 여부만으로는 공격 여부를 판단하기 어렵다.

따라서 Shell 자체를 탐지하는 것이 아니라, Shell을 부모 프로세스로 하여 실행되는 후속 명령을 함께 분석하는 방식으로 탐지 규칙을 설계한다.

이를 통해 Shell 이후 수행되는 명령의 종류와 목적을 함께 분석하여 정상적인 관리 작업과 공격자의 행위를 구분하도록 한다.

② Log Sources(Auditd에서 활용하는 로그)

MITRE ATT&CK에서는 Process Creation을 탐지하기 위한 다양한 데이터 소스를 제시하고 있다. 대표적으로 Auditd, Syslog, Osquery 등이 활용될 수 있으나, 프로젝트에서는 Linux Audit Framework(Auditd)를 기반으로 Rule Engine을 구현하였기 때문에 Auditd에서 제공하는 Process Creation 관련 로그만을 사용한다.

Auditd Record	활용 목적
SYSCALL	프로세스 생성 시 PID, PPID, UID, EUID, 실행 파일(exe) 등의 기본 정보를 수집한다.
EXECVE	실행된 프로그램의 명령행 인자(argv)를 수집한다.
CWD	프로세스가 실행된 현재 작업 디렉터리를 확인한다.
PATH	실행된 바이너리 및 관련 파일 경로를 확인한다.
PROCTITLE	원본 명령 문자열을 확인하여 디버깅 및 분석에 활용한다.

collector는 이러한 Auditd Record를 하나의 이벤트로 조립하고, Dispatcher는 이를 Rule Engine이 사용할 수 있는 SystemEvent 객체로 변환한다.

③ Mutable Elements(Rule Engine에서 사용하는 조건 변수)

MITRE ATT&CK에서는 Executable Name, Parent Process, Command Line Pattern 등 다양한 Mutable Element를 제시한다.

Auditd에서 수집가능한 정보를 기반으로 다음과 같은 Rule 조건을 정의한다.

Rule 조건	SystemEvent 필드	설명
ppid_exe	ParentImage	부모 프로세스가 Shell인지 확인한다. Dispatcher가 PPID를 이용하여 /proc/<PPID>/exe를 조회한 후 생성한 내부 분석 필드이다.
argv	CommandLine	EXECVE 레코드의 인자를 하나의 문자열로 결합한 명령행이다. 실행 이후 수행된 명령을 분석한다.

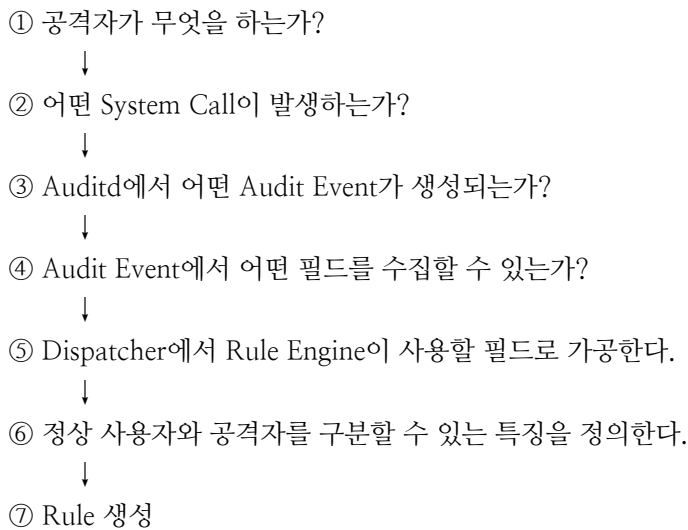
(5) Rule

① Rule 설계 방법

Mini EDR의 Rule은 특정 프로그램의 실행 여부만을 탐지하는 방식이 아니라, 공격자가 수행하는 행동(IOA, Indicator of Attack)을 기반으로 설계한다.

따라서 먼저 공격자가 수행하는 행위를 분석한 후, 해당 행위가 Linux Auditd에서 어떤 System Call과 Audit Event로 기록되는지를 확인한다. 이후 Audit Event에서 수집 가능한 필드 중 공격을 식별할 수 있는 정보를 추출하고, Dispatcher에서 Rule Engine이 사용할 수 있는 형태(SystemEvent)로 가공한 뒤 Rule을 생성한다.

Rule 설계 절차는 다음과 같다.



② AN1081 Rule 설계

AN1081은 Unix Shell을 이용하여 수행되는 공격 행위를 탐지하기 위한 Detection Strategy이다. 공격자는 시스템 침해 이후 대부분의 명령을 Unix Shell을 통해 실행하며, 이를 이용하여 시스템 정보를 수집(System Discovery), 파일 다운로드(Download), 네트워크 탐색(Network Discovery), Reverse Shell 생성, 지속성 확보 등의 후속 공격을 수행한다.

그러나 Linux 환경에서는 관리자와 일반 사용자 역시 Shell을 이용하여 다양한 작업을 수행하므로, 단순히 Shell의 실행 여부만으로는 공격 여부를 판단하기 어렵다.

따라서 Shell 자체를 탐지하는 것이 아니라 Shell의 부모 프로세스로 하여 실행되는 후속 명령을 함께 분석하는 방식으로 Rule을 설계한다.

이를 위해 Process Creation 이벤트에서 다음과 같은 정보를 사용한다.

Auditd Record	Dispatcher에서 생성한 필드	활용 목적
SYSCALL(PPID)	ParentImage	부모 프로세스가 Shell인지 확인
EXECVE	CommandLine	실행된 명령 및 인자 분석

Dispatcher에서 Auditd의 PPID를 이용하여 부모 프로세스의 실행 파일을 조회한 후 ParentImage 필드를 생성하며, EXECVE의 인자를 하나의 CommandLine 문자열로 결합한다.

Rule Engine은 ParentImage가 Shell 그룹에 속하는지 확인한 후, CommandLine에 포함된 명령이 Download, Sysattem Discovery, Network Discovery, Reverse Shell, Persistence 그룹에 속하는지를 순차적으로 검사하여 공격 여부를 판단한다.

③ Rule 생성

```
{
  "rules": [
    {
      "rule_id": "DET0384_AN1081",
      "name": "의심스러운 Unix Shell 활동의 행위 기반 탐지",
      "description": "프로세스 생성 이벤트를 기반으로 의심스러운 Unix Shell 실행 및 그에 따른 악성 활동을 탐지한다.",
      "event_type": "ProcessCreate",
      "syscall": ["execve", "execveat"],

      "conditions": {
        "ppid_exe": "Shell",
        "argv": {
          "download": "Download",
          "system_discovery": "SystemDiscovery",
          "network_discovery": "NetworkDiscovery",
          "reverse_shell": "ReverseShell",
          "persistence": "Persistence"
        }
      }
    }
  ]
}
```

Rule 조건	의미
ppid_exe	부모 프로세스가 Shell인지 확인한다.
argv: Download	파일 다운로드 명령을 탐지한다. curl, wget, ftp, scp, tftp
argv: SystemDiscovery	시스템 정보 수집 명령을 탐지한다. whoami, hostname, id, uname
argv: NetworkDiscovery	네트워크 탐색 명령을 탐지한다. ping, netstat, ip, ss
argv: ReverseShell	Reverse Shell 생성 명령을 탐지한다. nc, ncat, telnet, /dev/tcp
argv: Persistence	지속성 확보 명령을 탐지한다. crontab, systemctl, service

[7-3-2] Network Connect Rule

(1) System Call

NetworkConnect Event는 connect() System Call을 기반으로 생성한다.

① connect()란

클라이언트 프로그램 서버와 네트워크 통신을 시작하기 위해 연결을 요청할 때 사용하는 운영체제 함수이다. 소켓 디스크립터와 서버의 주소 정보를 기반으로 호출되며, 연결 지향형 통신(TCP)에서는 그 유명한 3-way handshake 과정을 유발한다.

```
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

- sockfd: socket() 시스템 콜로 생성된 소켓 파일 디스크립터이다.
- addr: 연결할 서버의 주소 정보(IP 주소와 포트 번호 등)를 담고 있는 구조체 포인터이다.
- addrlen: 두 번째 매개변수인 addr 구조체의 크기이다.

(2) MITRE ATT&CK - 탐지 전략

① Network Connection Creation

네트워크 연결 생성은 시스템이나 프로세스가 로컬 또는 원격 엔드포인트와 새로운 네트워크 세션을 시작하는 행위를 의미한다.

이 과정에서는 일반적으로 다음과 같은 네트워크 정보를 수집한다.

- 출발지(Source) IP 주소
- 목적지(Destination) IP 주소
- 출발지 및 목적지 포트 번호
- 사용된 프로토콜(TCP, UDP 등)
- 세션(Session) 메타데이터

이러한 이벤트를 지속적으로 모니터링하면 다음과 같은 악성 행위를 탐지하는 데 도움이 된다.

- Lateral Movement(횡적 이동): 공격자가 내부 시스템으로 이동하는 행위
- Data Exfiltration(데이터 유출): 중요 데이터를 외부로 전송하는 행위
- Command and Control(C2): 감염된 시스템 공격자의 서버와 통신하는 행위

데이터 수집 방법

Windows

- Event ID 5156(Filtering Platform Connection)
 - Windows Filtering Platform(WFP)을 통해 허용된 네트워크 연결을 기록한다.
- Sysmon Event ID 3(Network Connection Initiated)
 - 네트워크 연결이 시작될 때 다음 정보를 수집한다.
 - 실행할 프로세스
 - 출발지/목적지 IP
 - 포트 번호
 - 부모 프로세스(Parent Process)

Linux / macOS

- Netfilter(iptables), nftables 로그
 - 시스템의 들어오고 나가는 네트워크 연결을 기록한다.
- AuditD(connect 시스템 콜)
 - TCP, UDP, ICMP 등의 네트워크 연결 정보를 기록한다.
- Zeek(conn.log)
 - 네트워크 프로토콜, 연결 시간(Duration), 송수신 바이트 수(Bytes Transferred) 등의 정보를 수집한다.

클라우드 및 네트워크 인프라

- AWS VPC Flow Logs / Azure NSG Flow Logs
 - 클라우드 환경에서 IP 기반 네트워크 트래픽을 기록한다.
- Zeek(conn.log) / Suricata(Network Events)
 - 패킷 메타데이터를 수집하여 탐지 및 이벤트 상관 분석에 활용한다.

Endpoint Detection & Response(EDR)

EDR은 다음과 같은 비정상적인 네트워크 활동을 탐지한다.

- 새로운 C2(Command and Control) 서버와의 연결
- 데이터 유출(Data Exfiltration) 시도
- 비정상적인 외부 네트워크 통신

② Detection Strategy

웹 프로토콜 기반 C2 통신 및 웹 서비스 악용 탐지

- ID: DET0027
- 이름: Detection of Web Protocol-Based C2 Over HTTP, HTTPS, or WebSockets
- MITRE ID: T1071.001(Application Layer Protocol: Web Protocols)
- 설명: Linux 서버를 장악한 악성코드(비콘, 에이전트)가 외부 명령 제어(C2) 서버와 통신할 때 가장 많이 쓰는 방식이다. 일반적인 웹 브라우징이 일어나지 않는 Linux 서버 환경에서, 특정 프로세스나 스크립트가 외부의 생소한 IP/도메인으로 지속적인 HTTP/HTTPS 연결을 시도하는 행위를 잡아내는 필수 전략이다.

외부 툴 유입 및 다운로드 행위 체인 탐지

- ID: DET0060
- 이름: Detection Ingress Tool Transfers via Behavioral Chain
- MITRE ID: T1105(Ingress Tool Transfer)
- 설명: 공격자가 Linux 시스템의 취약점을 뚫고 들어온 직후, 후속 공격을 위해 외부에서 공격 툴(스캐너, 권한 상승 익스플로잇 등)을 받아오는 단계이다. Linux에서 curl, wget, python 등의 프로세스가 네트워크 연결을 수립한 직후 디스크에 실행 파일(ELF 등)을 쓰고 권한을 변경(chmod +x)하는 일련의 행위 체인을 탐지한다.

리버시 셸 및 유틸리티를 통한 터널링/프록시 탐지

- ID: DET0445
- 이름: Detection of Proxy Infrastructure Setup and Traffic Bridging
- MITRE ID: T1090(Proxy) / T1219(Remote Access Tools)
- 설명: 내부망 방화벽을 우회하기 위해 Linux 서버를 프록시 기지나 내부 릴레이 징검다리로 악용하는 행위이다. Chisel, Ligolo-ng 같은 툴이나 nc(netcat), socat을 이용해 외부와 세션을 유지하는 행위, 혹은 개발용 IDE 툴의 원격 터널링 기능(VS Code Tunnel 등)이 비정상적인 네트워크 연결을 생성할 때 이를 프로세스 아규먼트와 결합해 탐지한다.

DNS 터널링 및 데이터 유출 탐지

- ID: DET0400
- 이름: Behavioral Detection of DNS Tunneling and Application Layer Abuse
- MITRE ID: T1071.004(Application Layer Protocol: DNS)
- 설명: 엄격한 아웃바운드 방화벽 정책이 적용된 Linux 환경이라도 DNS(UDP53) 쿼리는 허용되는 경우가 많다. 공격자는 이를 악용해 DNS 질의 패킷 내에 명령어나 유출 데이터를 숨어 보낸다. Linux 내부 네임서버 설정이나 프로세스가 비정상적으로 대량의 서브도메인(예: 서브도메인.악성도메인.com)으로 DNS 연결 및 쿼리를 생성하는 패턴을 추적한다.

비정상 포트를 이용한 리액티브 C2 통신 탐지

- ID: DET0227
- 이름: Detection Strategy for Non-Standard Ports
- MITRE ID: T1571(Non-Standard Port)
- 설명: Linux 서버의 일반적인 네트워크 통신은 정해진 포트(80, 443, 53, 22 등)를 통해 이루어진다. 하지만 공격 코드가 C2와 통신하거나 데이터를 유출할 때 방화벽 모니터링을 피하기 위해 임의의 비정상 포트(예: 4444, 8080, 9001 등)로 아웃바운드 커넥션을 생성하는 경우가 많다. 프로세스의 목적과 맞지 않는 포트 연결을 탐지하는 표준적인 전략이다.

(3) MITRE ATT&CK – 공격 방법

MITRE ATT&CK에서는 Network Connection 이벤트를 활용한 다양한 Detection Strategy를 제공한다. 그러나 모든 탐지 전략을 구현하기에는 프로젝트의 범위와 개발 기간에 한계가 있으므로, 이 프로젝트에서는 Linux 환경에서 악성코드의 C2(Command & Control) 통신에 가장 빈번하게 악용되는 웹 프로토콜 기반 통신을 탐지하기 위해 Application Layer Protocol: Web Protocols(T1071.001)를 Rule Engine의 탐지 규칙으로 선정하였다.

T1071.001 – Application Layer Protocol: Web Protocols

공격자는 정상적인 웹 트래픽에 섞여 탐지를 회피하거나 네트워크 필터링을 우회하기 위해 웹 애플리케이션 계층(Application Layer)의 통신 프로토콜을 사용할 수 있다.

원격 시스템으로 전달되는 명령(Command)과, 그 명령의 실행 결과(Result)는 대부분 클라이언트와 서버 간에 오가는 프로토콜 데이터 안에 숨겨져 전송된다.

대표적으로 HTTP/HTTPS와 WebSocket은 대부분의 네트워크 환경에서 매우 흔하게 사용되는 웹 통신 프로토콜이다. 이러한 프로토콜은 다양한 헤더(Header)와 필드를 가지고 있기 때문에, 공격자는 그 안에 데이터를 은닉할 수 있다.

공격자는 이러한 프로토콜을 악용하여 피해자 네트워크 내부의 감염된 시스템과 자신이 제어하는 서버(Command & Control 서버) 간에 통신을 수행하면서도, 정상적인 웹 트래픽처럼 위장하여 탐지를 어렵게 만들 수 있다.

(4) Detection Strategy 분석

① AN0076

AN0076은 HTTP, HTTPS, WebSocket과 같은 웹 프로토콜을 이용하여 수행되는 Command and Control(C2) 통신을 탐지하기 위한 분석 기법이다.

Linux 서버를 침해한 공격자는 일반적인 웹 트래픽으로 위장하기 위해 HTTP 또는 HTTPS 프로토콜을 이용하여 외부 C2 서버와 지속적으로 통신한다. 이러한 통신은 정상적인 웹 요청과 유사하기 때문에 단순히 네트워크 연결이 발생했다는 사실만으로는 공격 여부를 판단하기 어렵다.

따라서 네트워크 연결을 생성한 프로세스와 연결 대상 정보를 함께 분석하는 방식으로 Rule을 설계한다. Process Create 이벤트에서 수집한 프로세스 정보와 NetworkConnect 이벤트에서 수집한 목적지 IP, 목적지 Port, 프로토콜 정보를 함께 활용하여 의심스러운 외부 연결을 탐지한다.

② Log Sources(Auditd에서 활용하는 로그)

MITRE ATT&CK에서는 네트워크 연결을 탐지하기 위해 다양한 데이터 소스를 제시한다. 대표적으로 Zeek, Osquery, Netfilter 등의 네트워크 로그를 사용할 수 있으나, 프로젝트에서 Linux Audit Framework(Auditd)를 기반으로 Rule Engine을 구현하였으므로 Auditd에서 생성되는 Network Connection 관련 로그만을 사용한다.

Network Connect Rule에서 활용하는 Auditd Record는 다음과 같다.

Auditd Record	활용 목적
SYSCALL	connect() 시스템 콜 발생 여부와 프로세스 정보를 수집한다.
SOCKADDR	목적지 IP 주소와 목적지 Port 정보를 수집한다.
EXECVE	네트워크 연결을 생성한 프로세스의 명령행을 수집한다.

③ Mutable Elements(Rule Engine에서 사용하는 조건 변수)

Rule 조건	SystemEvent 필드	설명
exe	ImagePath	네트워크 연결을 생성한 프로세스를 확인한다.
argv	CommandLine	URL, HTTP 옵션, 다운로드 명령 등을 분석한다.
dest_ip	DstIP	연결 대상 IP 주소를 확인한다.
dest_port	DstPort	연결 대상 Port를 확인한다.
protocol	Protocol	TCP 또는 UDP 프로토콜 여부를 확인한다.

(5) Rule

① AN0076 Rule 설계

Mini EDR의 NetworkConnect Rule은 단순히 외부 네트워크 연결이 발생했다는 사실만으로 탐지하지 않는다. 정상적인 Linux 서버 역시 패키지 설치, 시스템 업데이트, API 호출 등 다양한 이유로 외부와 통신하기 때문이다.

따라서 어떤 프로세스가, 어디로, 어떤 방식으로 연결을 생성했는지 함께 분석하도록 Rule을 설계한다. 이를 위해 Auditd에서 수집한 정보를 Dispatcher가 Rule Engine에서 사용할 수 있는 필드로 변환하여 다음과 같이 활용한다.

Auditd Record	Dispatcher에서 생성한 필드	활용 목적
SYSCALL	ParentImage	네트워크 연결을 생성한 실행 파일 확인
EXECVE	CommandLine	URL, HTTP 옵션 및 다운로드 명령 분석
SOCKADDR	DstIP	목적지 IP 확인
SOCKADDR	DstPort	목적지 Port 확인
SOCKADDR	Protocol	TCP/UDP 프로토콜 확인

Rule Engine은 위 정보를 기반으로 HTTP Client(curl, wget 등)의 실행 여부와 목적지 정보를 함께 분석하여 C2 통신 가능성을 판단한다.

② Rule 생성

```
{
  "rules": [
    {
      "rule_id": "DET0027_AN0076",
      "name": "HTTP, HTTPS 또는 웹 프로토콜을 이용한 웹 프로토콜 기반 C2 탐지",
      "description": "HTTP/HTTPS/WebSocket 기반 C2 통신을 탐지한다.",
      "event_type": "NetworkConnect",
      "syscall": "connect",

      "conditions": {
        "exe": "HttpClient",
        "argv": {
          "download": "Download"
        },
        "protocol": "WebProtocol",
        "dest_port": "AbnormalPort"
      }
    }
  ]
}
```

[7-3-3] File Create Rule

(1) System Call

FileCreate Event는 creat() System Call을 기반으로 생성된다.

creat()란

리눅스 및 유닉스 환경에서 새로운 파일을 생성하거나 기존 파일을 초기화하여 쓰기 전용으로 열 때 사용하는 시스템 콜이다.
open() 시스템 콜에 O_CREAT | O_WRONLY | O_TRUNC 플래그를 조합한 것과 동일한 역할을 한다.

```
int creat(const char *pathname, mode_t mode);
```

- pathname: 생성할 파일의 경로와 이름(문자열)
- mode: 파일 접근 권한(예: 0644는 소유자 읽기/쓰기, 그룹/기타 읽기 가능)

(2) MITRE ATT&CK - 탐지 전략 종류

① File Creation

새로운 파일이 시스템 또는 네트워크 저장소에 생성되는 행위를 의미한다.
이러한 이벤트는 문서 저장, 데이터 기록, 파일 배포 등과 같은 정상적인 작업을 나타낼 수도 있지만, 악성 파일 생성과 같은 악의적인 행위의 징후일 수도 있다.

따라서 파일 생성 이벤트를 기록하여 정상적인 파일 생성 활동과 잠재적인 악성 행위를 식별하는 데 도움이 된다. 대표적인 예로 Sysmon Event ID 11 또는 Linux Auditd의 파일 생성 로그가 있다.

② Detection Strategy

웹셸(Web Shell) 생성 및 실행 체인 탐지

- ID: DET0394
- 이름: Web Shell Detection via Server Behavior and File Execution Chains
- MITRE ID: T1505.003(Server Software Component: Web Shell)
- 설명: Linux 기반 웹 서버 취약점 공격의 최종 목적지는 대부분 웹셸 설치이다. 웹 서비스의 업로드 경로(uploads/, images/ 등)나 웹 루트 디렉토리에 전형적인 텍스트/스크립트 형태의 파일(.php, .jsp, .py)이 비정상적으로 생성되는 순간을 잡고, 이것이 웹 서버 프로세스에 의해 실행되는 체인을 모니터링하는 절대적인 필수 전략이다.

공급망 공격 및 악성 라이브러리 설치 탐지

- ID: DET0252
- 이름: User-Initiated Malicious Library Installation via Package Manager(T1204.005)
- MITRE ID: T1204.005(User Execution: Malicious Image / Malicious Library)
- 설명: 최근 Linux 환경을 겨냥한 가장 핫하고 위험한 공격 형태이다. 오픈소스 패키지 매니저(pip, npm, apt, yum 등)를 통해 타이포스쿼팅(이름 오타 유도)이나 침해된 패키지를 다운로드하여 시스템 내에 악성 라이브러리 파일들을 생성, 오용하는 행위를 탐지한다.

커널 모듈(Rootkit) 및 드라이버 등록을 통한 영속성 탐지

- ID: DET0450
- 이름: Detection Strategy for Kernel Modules and Extensions Autostart Execution
- MITRE ID: T1547.006(Boot or Logon Autostart Execution: Kernel Modules and Extensions)
- 설명: 고도화된 Linux 공격자들은 시스템 명령어를 변조하거나 백신을 위회하기 위해 커널 레벨 루트킷을 심는다. /lib/modules/ 나 /usr/lib/modules/ 디렉토리에 새로운 커널 모듈 파일(.ko)이 드롭되거나, insmod, modprobe 명령어를 통해 파일 시스템 변화가 일어나는 행위를 엄격하게 감시해야 한다.

XDG Autostart 및 기동 스크립트를 이용한 자동 실행 탐지

- ID: DET0390
- 이름: Linux Detection Strategy for T1547.013-XDG Autostart Entries
- MITRE ID: T1547.013(Boot or Logon Autostart Execution:XDG Autostart Entries)
- 설명: Linux(특히 데스크톱 환경이나 특정 서버 환경)에서 사용자가 로그인할 때 악성코드가 자동으로 켜지도록 만드는 기법이다. /etc/xdg/autostart/, ~/.config/autostart/ 경로에 의심스러운 .desktop 파일이 생성되거나 수정되는 패턴을 추적하여 공격자의 지속성 확보 기법을 무력화한다.

시스템 서비스(Systemd) 등록 및 유닛 파일 변조 탐지

- ID: DET0253
- 이름: Detection of Systemd Service Creation or Modification on Linux
- MITRE ID: T1543.002(Create or Modify System Process: Systemd Service)
- 설명: Linux 환경에서 가장 대중적인 백도어 구성 방식이다. 시스템 재부팅 후에도 악성코드가 관리자 권한으로 항상 실행되도록 하기 위해 /etc/systemd/system/ 경로에 새로운 서비스 유닛 파일(.service)을 생성하거나 기존 정상 서비스를 변조하는 행위를 실시간으로 감시하는 전략이다.

(3) MITRE ATT&CK - 공격 방법

Create or Modify System Process: Systemd Service

공격자는 Systemd 서비스를 생성하거나 기존 서비스를 수정하여 악성코드를 지속적으로 실행함으로써 시스템 내 영속성을 확보할 수 있다.

Systemd는 Linux 환경에서 백그라운드 서비스(데몬)와 시스템 자원을 관리하는 기본 초기화(init) 시스템으로, 대부분의 Linux 배포판에서 사용된다.

Systemd는 .service 확장자를 갖는 Unit 파일을 이용하여 서비스의 실행 정보를 관리한다.

일반적으로 시스템 단위 서비스는 /etc/systemd/system/ 또는 /usr/lib/systemd/system/ 등에 저장되며, 사용자 단위 서비스는 사용자 홈 디렉터리의 ~/.config/systemd/user/ 등에 저장된다.

.service 파일에는 서비스 실행 방식을 정의하는 다양한 지시어(Directive)가 존재한다.

대표적으로 ExecStart, ExecStartPre, ExecStartPost 는 서비스 시작 시 실행할 명령을 정의하며, ExecReload는 서비스 재작 시, ExecStop, ExecStopPre, ExecStopPost는 서비스 종료 시 실행할 명령을 지정한다.

공격자는 새로운 .service 파일을 생성하거나 기존 서비스의 ExecStart 등의 지시어를 수정하여 악성 프로그램이 실행되도록 설정할 수 있다. 또한, User 지시어를 변경하여 특정 권한으로 서비스를 실행하거나, 심볼릭 링크(Symbolic Link)를 이용해 실제 악성 파일의 위치를 숨기는 기법도 사용할 수 있다.

이러한 방식은 시스템이 재부팅된 이후에도 악성코드가 자동으로 실행되도록 하여 장기간 시스템에 은닉하는 영속성 확보 기법으로 활용된다.

(4) Detection Strategy 분석

① AN0701

AN701은 Linux 환경에서 Systemd 서비스 유닛 파일이 생성되거나 수정되는 행위를 탐지하기 위한 분석 기법이다.

Systemd 서비스는 시스템 부팅 이후 특정 프로그램을 자동으로 실행하거나 백그라운드 서비스로 유지하는데 사용된다. 공격자는 이를 악용하여 /etc/systemd/system/, /lib/systemd/system/, 사용자 단위 systemd 경로 등에 악성 .service 파일을 생성함으로써 재부팅 이후에도 악성 프로그램이 지속적으로 실행되도록 만들 수 있다.

복잡한 ExecStart 내용 분석이나 파일 엔트로피 분석까지는 수행하지 않고, Ssystemd 서비스 파일이 생성되는 경로 기반 행위를 우선 탐지 대상으로 선정한다. 즉, Auditd의 파일 생성 이벤트를 이용하여 .service 파일이 시스템 또는 사용자 서비스 경로에 생성되는지를 확인하고, 이를 Persistence 확보 가능성이 있는 행위로 판단한다.

② Log Sources(Auditd에서 활용하는 로그)

FileCreate 이벤트를 탐지하기 위해 Linux Audit Framework(Auditd)에서 생성되는 파일 생성 관련 로그를 사용한다.

Auditd Record	활용 목적
SYSCALL	creat() 시스템 콜 발생 여부와 프로세스 정보를 수집한다.
PATH	생성되거나 수정된 파일의 경로를 수집한다.
CWD	파일 생성 당시의 작업 디렉터리를 확인한다.
PROCTITLE	원본 명령 문자열을 디버깅 및 분석에 활용한다.

Collector는 동일 Auditd ID를 기준으로 위 Record들을 하나의 AuditdLogGroup으로 조립하고, Dispatcher는 PATH.name 값을 SystemEvent.PathName 또는 TargetFile로 변환한다.

③ Mutable Elements(Rule Engine에서 사용하는 조건 변수)

Rule 조건	SystemEvent 필드	설명
path	PathName	생성된 파일 경로가 Systemd 서비스 경로에 해당하는지 확인한다.
file_ext	FileExt	생성된 파일의 확장자가 .service인지 확인할 수 있다.
exe	ImagePath	파일 생성을 수행한 프로세스를 확인한다.
argv	CommandLine	파일 생성 명령 또는 관련 인자를 분석한다.

현재 1차 Rule에서는 path 조건만 사용한다.

이후 단계에서는 exe, argv, file_ext를 함께 사용하여 systemctl, tee, echo, vim 등을 통한 서비스 파일 생성 행위를 더 정밀하게 분석할 수 있다.

(5) Rule

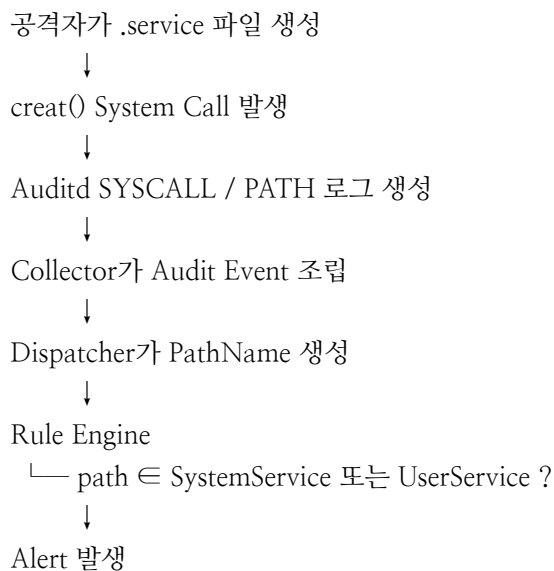
① AN0701 Rule 설계

AN0701 Rule은 Systemd 서비스 유닛 파일 생성 행위를 탐지하기 위해 설계한다.

공격자는 지속성 확보를 위해 .service 파일을 생성하고, 해당 파일 안에 악성 실행 경로를 ExecStart로 등록할 수 있다. 이 방식은 시스템 재부팅 이후에도 악성 프로그램이 자동으로 실행되도록 만들 수 있기 때문에 Linux 환경에서 중요한 Persistence 탐지 대상이다.

Auditd의 creat() 시스템 콜과 PATH 레코드를 이용하여 생성된 파일의 경로를 수집한다.

이후 Dispatcher는 PATH.name 값을 SystemEvent.PathName으로 변환하고, Rule Engine은 이 값이 Systemd 서비스 경로 패턴에 해당하는지 검사한다.



② Rule 생성

```
{
  "rules": [

    {
      "rule_id": "DET0253_AN0701",
      "name": "Linux에서의 Systemd 서비스 생성 또는 수정 탐지",
      "description": "시스템 또는 사용자 수준의 디렉터리에서 .service 유닛 파일이 생성되거나 수정되는 행위를 탐지한다.",
      "event_type": "FileCreate",
      "syscall": "creat",

      "conditions": {
        "path": [
          "SystemService",
          "UserService"
        ]
      }
    }
  ]
}
```

[7-3-4] File Delete Rule

(1) System Call

File Delete Event는 unlink()와 unlinkat() System Call을 기반으로 생성된다.

① unlink()란

지정된 경로명에 해당하는 파일의 하드 링크를 제거하고, 해당 파일에 대한 링크 카운트(link count)를 1감소시킨다. 링크 카운트가 0이되고 해당 파일을 열고 있는 프로세스가 없을 경우, 파일 시스템에서 실제 데이터가 삭제된다.

```
int unlink(const char *pathname);
```

- pathname: 삭제할 파일의 경로 및 이름을 가리키는 문자열 포인터이다.

② unlinkat()

디렉터리 파일 설명자(file descriptor)를 기준으로 파일이나 디렉터리를 삭제할 때 사용하는 시스템 콜이다. 기존의 unlink()나 rmdir() 시스템 콜과 동일한 역할을 하지만, 기준 경로를 지정할 수 있어 상대 경로 처리에 유용하다.

```
int unlinkat(int dirfd, const char *pathname, int flags);
```

- dirfd: 기준이 되는 디렉터리의 파일 설명자이다. AT_FDCWD를 전달하면 현재 작업 디렉터리를 기준으로 pathname을 해석한다.

- pathname: 삭제할 대상 파일이나 디렉터리의 경로이다.

- flags: 연산의 방식을 결정하는 비트마스크이다.

- 0: 일반 파일이나 링크를 삭제할 때 사용되며 unlink()와 동일하게 동작한다.

- AT_REMOVEDIR: 디렉터리를 삭제할 때 사용되며 rmdir()과 동일하게 동작한다.

(2) MITRE ATT&CK - 탐지 전략 종류

① File Deletion

시스템 또는 저장 장치에서 파일이 삭제되는 행위를 의미한다. 이러한 이벤트는 정상적인 파일 정리(Housekeeping) 작업의 일부일 수도 있지만, 공격자가 자신의 흔적을 은폐하기 위해 파일을 삭제하는 악의적인 행위일 수도 있다. 따라서 파일 삭제 이벤트를 모니터링하면 무단 파일 삭제나 의심스러운 활동을 식별하는 데 도움이 된다.

② Detection Strategy

Linux 로그 파일 삭제 및 변조(안티 포렌식) 탐지)

ID: DET0520

이름: Behavioral Detection of Log File Clearing on Linux and macOS

MITRE ID: T1685.006(Indicator Removal: Log File Clearing)

설명: Linux 서버에 무단 침입한 공격자가 가장 먼저 수행하는 방어 우회(Defence Evasion) 행위이다. 흔적을 지우기 위해 /var/log/auth.log, /var/log/secure, /var/log/message 등 핵심 시스템 로그 파일을 rm -rf로 삭제하거나, echo "" > [파일 경로] 형태로 비우는 행위를 실시간으로 모니터링하는 필수 전략이다.

셸 명령 이력 삭제(Command History Clearing) 탐지

ID: DET0165

이름: Behavioral Detection of Command History Clearing

MITRE ID: T1070.003(Indicator Removal: Clear Command History)

설명: 공격자가 Linux 터미널에서 수행한 악성 행위(백도어 다운로드, 권한 상승 등)를 감추기 위해 사용한다. `history -c` 명령어로 현재 세션 히스토리를 통째로 날리거나 `shred -u ~/.bash_history`를 통해 히스토리 저장 파일을 완전히 파쇄하는 행위, 혹은 `unset HISTFILE`로 기록 자체를 비활성화하는 행위를 탐지한다.

대량 파일 파괴 및 랜섬웨어(Mass Deletion) 행위 탐지

ID: DET0416

이름: Detection of Data Destruction Across Platforms via Mass Overwrite and Deletion Patterns

MITRE ID: T1485(Data Destruction)

설명: 단순한 흔적 지우기를 넘어 기업에 타격을 입히려는 목적(디펜딩, 랜섬웨어, 파괴형 악성코드)의 행위이다. 특정 단시간 내에 데이터베이스 디렉토리나 주요 사용자 디렉토리(`~/*`, `/home/*`, `/data/*`)에서 수많은 파일이 동시다발적으로 삭제(`unlink`)되거나 의미 없는 데이터로 덮어쓰기되는 비정상 패턴을 잡아낸다.

백업 기능 및 시스템 복구 무력화 탐지

ID: DET0329

이름: Behavioral Detection for T1490 - Inhibit System Recovery

MITRE ID: T1490(Inhibit System Recovery)

설명: 랜섬웨어 공격자들이 피해자가 시스템을 원상복구하지 못하도록 사전에 방해하는 기법이다. Linux 환경에서는 로컬 백업 본이나 스냅샷을 관리하는 서비스(예: LVM 스냅샷 삭제, `rsnapshot` 아카이브 삭제 등)를 강제로 지우거나 무력화하는 명령어를 감시한다.

핵심 서비스 강제 종료(Service Stop) 탐지

ID: DET0021

이름: Behavioral Detection for Service Stop across Platforms

MITRE ID: T1489(Service Stop)

설명: 공격자가 시스템 보안 솔루션(EDR 에이전트, 백신 등)의 모니터링을 피하거나, 데이터베이스 각동을 중지시켜 파일을 안전하게 잠그거나 탈취하기 위해 수행한다.

`systemctl stop`, `kill -9` 프로세스 시그널 등을 이용해 지정된 화이트리스트 외의 필수 데몬(Service)들이 비정상 종료되는 이벤트를 추적한다.

(3) MITRE ATT&CK - 공격 방법

T1685.006 - Disable or Modify Tools: Clear Linux or Mac System Logs

공격자는 침입 흔적을 은폐하기 위해 시스템 로그를 삭제할 수 있다. macOS와 Linux는 시스템 또는 사용자가 수행한 다양한 활동을 시스템 로그에 기록하며, 대부분의 기본 로그는 /var/log/ 디렉터리에 저장된다.

/var/log/ 디렉터리의 하위 디렉터리와 파일은 기능별로 로그를 구분하여 관리하며, 대표적인 예는 다음과 같다.

- /var/log/messages: 시스템 전반에서 발생하는 일반적인 메시지 및 시스템 관련 로그
- /var/log/secure 또는 /var/log/auth.log: 사용자 인증(Authentication) 관련 로그
- /var/log/utmp 또는 /var/log/wtmp: 사용자 로그인 및 로그아웃 기록
- /var/log/kern.log: Linux 커널(Kernel) 관련 로그
- /var/log/cron.log: Cron 작업 실행 로그
- /var/log/maillog: 메일 서버 관련 로그
- /var/log/httpd: 웹 서버(HTTPD)의 접근(Access) 및 오류(Error) 로그

공격자는 이러한 로그 파일을 삭제하거나 수정하여 자신의 침입 흔적을 숨기고, 보안 관리자의 분석과 포렌식 조사를 방해하려고 한다. 따라서 시스템 로그 파일의 삭제 또는 비정상적인 수정 행위를 지속적으로 모니터링하는 것은 공격자의 방어 회피 기법을 탐지하는 데 중요한 요소이다.

(4) Detection Strategy 분석

① AN1438

AN1438은 Linux/macOS 환경에서 시스템 로그 파일이 삭제되거나 비정상적으로 정리되는 행위를 탐지하기 위한 분석 기법이다.

공격자는 침입 이후 자신의 흔적을 숨기기 위해 /var/log/auth.log, /var/log/secure, /var/log/message, /var/log/wtmp와 같은 인증 로그, 시스템 로그, 로그인 기록 파일을 삭제하거나 초기화하려고 한다. 이러한 행위는 보안 관리자의 사고 분석과 포렌식 조사를 방해하는 대표적인 Defense Evasion 기법이다.

1차 구현 범위에서 로그 파일의 내용 초기화(truncate)나 시간 상관 분석까지는 수행하지 않고, Auditd의 파일 삭제 이벤트를 기반으로 로그 파일이 삭제되는 행위를 탐지 대상으로 선정한다.

② Log Sources(Auditd에서 활용하는 로그)

FileDelete 이벤트를 탐지하기 위해 Linux Auditd Framework(Auditd)에서 생성되는 파일 삭제 관련 로그를 사용한다.

Auditd Record	활용 목적
SYSCALL	unlink() 또는 unlinkat() 시스템 콜 발생 여부와 프로세스 정보를 수집한다.
PATH	삭제 대상 파일의 경로를 수집한다.
CWD	삭제 명령이 수행된 현재 작업 디렉터리를 확인한다.
PROCTITLE	원본 명령 문자열을 디버깅 및 분석에 활용한다.

Collector는 동일 Audit ID를 기준으로 위 Record들을 하나의 AuditdLogGroup으로 조립하고, Dispatcher는 PATH.name 값을 SystemEvent.PathName 또는 TargetFile로 변환한다.

③ Mutable Elements(Rule Engine에서 사용하는 조건 변수)

Rule 조건	SystemEvent 필드	설명
pathname	PathName	삭제 대상 파일 경로가 주요 로그 파일 경로에 해당하는지 확인한다.
exe	ImagePath	삭제 행위를 수행한 프로세스를 확인한다.
argv	CommandLine	rm, shred, truncate 등 삭제 명령 및 인자를 분석할 수 있다.
euid	EUID	root 권한으로 로그 삭제가 수행되었는지 확인할 수 있다.

현재 1차 Rule에서 pathname 조건만 사용한다.

이후 확장 단계에서는 exe, argv, euid, 일정 시간 내 반복 삭제 횟수 등을 함께 사용하여 로그 삭제 행위를 더 정밀하게 탐지할 수 있다.

(5) Rule

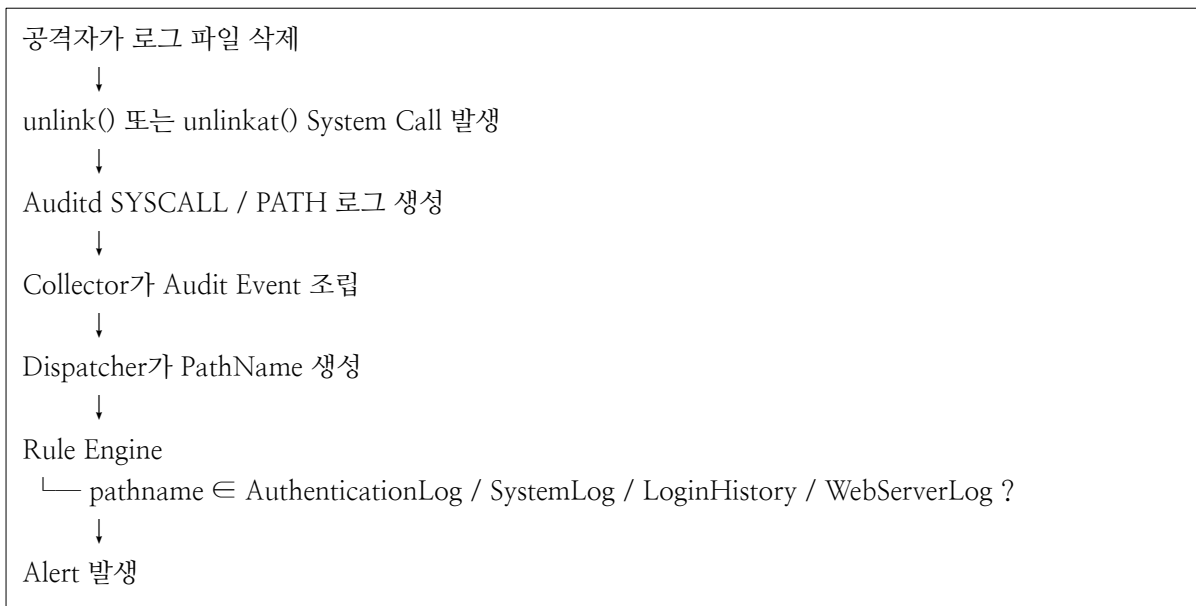
① AN1438 Rule 설계

AN1438 Rule은 공격자의 흔한 은폐 행위 중 하나인 시스템 로그 파일 삭제를 탐지하기 위해 설계한다.

Linux 시스템에서는 인증 로그, 시스템 로그, 로그인 이력, 웹 서버 로그 등이 /var/log/ 하위에 저장된다. 공격자는 침입 이후 자신이 실행할 명령, 로그인 기록, 인증 실패 흔적, 웹 서버 접근 흔적을 숨기기 위해 이러한 로그 파일을 삭제하거나 정리하려고 할 수 있다.

Auditd의 unlink() 및 unlinkat() 시스템 콜과 PATH 레코드를 이용하여 삭제 대상 파일의 경로를 수집한다. 이후 Dispatcher는 PATH.name 값을 SystemEvent.PathName으로 변환하고, Rule Engine은 해당 경로가 주요 로그 파일 그룹에 포함되는지 검사한다.

탐지 흐름은 다음과 같다.



② Rule 생성

```
{
  "rules": [
    {
      "rule_id": "DET0520_AN1438",
      "name": "Behavioral Detection of Log File Clearing on Linux and macOS",
      "description": "/var/log/ 디렉터리의 시스템 로그를 대상으로 수행되는 의심스러운 명령 실행과 로그 파일 삭제 또는 내용 초기화 행위를 탐지한다.",
      "log_source": "file_delete",
      "conditions": {
        "pathname": [
          "AuthenticationLog",
          "SystemLog",
          "LoginHistory",
          "WebServerLog"
        ]
      }
    }
  ]
}
```

[7-3-5] ProcessAccess

(1) System Call

ProcessAccess Event는 ptrace System Call을 기반으로 생성한다.

① ptrace란

한 프로세스(트레이서)가 다른 프로세스(트레이시)의 실행을 관찰하고 제어할 수 있게 해주는 유닉스 계열의 시스템 콜이다. 프로세스 메모리 및 레지스터 읽기/쓰기, 시스템 콜 가로채기, 시그널 처리 등을 지원하여 주로 디버거(gdb)나 시스템 분석 도구(strace)의 핵심 기술로 사용된다.

동작 원리: 트레이서 프로세스가 ptrace(PTRACE_ATTACH, pid, ...)를 호출해 대상 프로세스를 제어하겠다고 선언하면 대상은 멈춘다. 이후 상태 확인 및 레지스터 조작 등을 수행한 후 PTRACE_CONT 등으로 다시 실행을 재개시킨다.

- PTRACE_PEEKTEXT / PTRACE_PEEKDATA: 대상 프로세스의 메모리 읽기
- PTRACE_PEEKTEXT / PTRACE_PEEKDATA: 대상 프로세스의 메모리 쓰기 및 코드/데이터 조작
- PTRACE_SYSCALL: 대상 프로세스가 시스템 콜을 호출하거나 반환할 때마다 실행을 가로채기

(2) MITRE ATT&CK - 탐지 전략

① Process Access

프로세스 접근(Process Access)은 하나의 프로세스가 다른 프로세스를 열거나(Open) 접근을 시도하는 행위를 의미한다. 이러한 접근은 대상 프로세스의 메모리를 조회하거나 조작하고, 핸들(Handle)에 접근하거나, 실행 흐름을 변경하기 위해 수행될 수 있다.

프로세스 접근 이벤트를 모니터링하면 디버깅(Debugging), 프로세스 간 통신(Inter-Process Communication IPC)과 같은 정상적인 행위뿐만 아니라, 프로세스 인젝션(Process Injection)과 같은 악성 행위를 식별하는 데 중요한 정보를 제공한다.

데이터 수집 방법(Data Collection Measures)

1. EndPoint Detection and Response(EDR) 도구

프로세스 간 접근 및 메모리 조작과 관련된 다양한 텔레메트리(Telemetry)를 수집하는 EDR 솔루션을 활용한다.

2. Sysmon(Windows)

Event ID 10을 통해 프로세스 접근 시도를 기록하며, 다음과 같은 정보를 제공한다.

- 접근을 시도한 프로세스(Source Process)
- 접근 대상 프로세스(Target Process)
- 요청된 접근 권한(Access Rights)
- 프로세스 ID(Process ID) 정보

3. Windows Event Log

- Event ID 4656: 시스템 객체에 대한 접근 시도를 기록한다.
- Event ID 4690: 프로세스 수정(Process Modification) 시도를 기록하며, 비인가된 프로세스 변경 행위를 탐지하는 데 활용된다.

4. Linux/macOS 모니터링=

- Auditd: ptrace, open, read, write 등의 시스템 호출(Syscall)을 추적하여 프로세스 접근 행위를 모니터링한다.
- eBPF/XDP: 커널 수준에서 프로세스 접근을 저수준으로 모니터링한다.
- OSQuery: SQL과 유사한 질의를 이용하여 프로세스 접근 관련 정보를 수집한다.

5. Procmon 및 디버깅 도구

- Windows Procmon(Process Monitor): 프로세스 간 상호작용을 실시간으로 모니터링한다.
- Linux strace / ptrace : 시스템 호출 수준에서 프로세스의 동작과 접근 행위를 추적하는데 사용된다.

② Detection Strategy

/proc 파일시스템 접근을 통한 자격 증명 탈취 탐지

- ID: DET0593
- 이름: Detecting OS Credential Dumping via /proc Filesystem Access on Linux
- MITRE ID: T1003.007(OS Credential Dumping: /proc Filesystem)
- 설명: Linux 환경에서 공격자가 일반 사용자나 프로세스의 메모리에 남아있는 비밀번호, API 키, 인증 토큰을 덤프할 때 가장 먼저 접근하는 곳이 /proc이다. 특정 프로세스가 자기 자신이 아닌 다른 프로세스의 메모리 영역(/proc/[PID]/mem, /proc/[PID]/maps)에 접근하여 읽기(Read) 작업을 수행하는 행위를 감시하는 핵심 전략이다.

Ptrace 및 시스템 콜 기반 프로세스 인젝션 탐지

- ID: DET0508
- 이름: Behavioral Detection of Process Injection Across Platforms
- MITRE ID: T1055(Process Injection)
- 설명: 공격자가 탐지를 회피하기 위해 정상적인 시스템 프로세스(예: sshd, systemd)에 악성 코드를 주입하는 기법이다. Linux에서는 주로 ptrace 시스템 콜을 사용해 타겟 프로세스를 제어하거나 메모리를 수정하므로, 권한이 없는 프로세스가 비정상적으로 PTRACE_ATTACH 또는 PTRACE_POKETEXT 같은 API 호출을 발생시키는 행위를 탐지한다.

브라우저 및 웹 세션 쿠키 탈취 탐지

- ID: DET0509
- 이름: Detection of Web Session Cookie Theft via File, Memory, and Network Artifacts
- MITRE ID: T1539(Streal Web Session Cookie)
- 설명: Linux 데스크톱 환경이나 개발자 자산(VDI)을 공격할 때 매우 빈번하게 발생한다. 공격자는 MFA(멀티팩터 인증)를 우회하기 위해 크롬이나 파이어폭스 같은 브라우저 프로세스의 메모리나 로컬 데이터베이스 파일(Cookies, Login Data)에 직접 접근하여 세션 쿠키를 훔치려 한다. 이 메모리 및 특정 경로 파일 접근 시도를 모니터링한다.

직접 시스템 콜(Direct Syscall) 및 비정상 API 호출 탐지

- ID: DET0529
- 이름: Behavioral Detection of Native API Invocation via Unusual DLL Loads and Direct Syscalls
- MITRE ID: T1106(Native API)
- 설명: 보안 솔루션(EDR)의 표준 API 후킹(Hooking)을 우회하기 위해, 공격자가 정상적인 glibc 라이브러리를 거치지 않고 커널에 직접 시스템 콜(Direct Syscall)을 날리는 고도화된 기법이다. 혹은 프로세스의 정상적인 실행 흐름과 무관한 생소한 바이너리에서 커널 레벨 API가 연속적으로 호출되는 비정상 패턴을 잡아낸다.

가상화 및 샌드박스 우회(Evasion)를 위한 시스템 체크 탐지

- ID: DET0168
- 이름: Virtualization/Sandbox Evasion via System Checks across Windows, Linux, macOS
- MITRE ID: T1497.001(Virtualization/Sandbox Evasion: System Checks)
- 설명: 악성코드가 자신이 분석 환경(샌드박스, VM)에 있는지 확인하기 위해 Linux API 및 메모리 정보를 조회하는 행위이다. dmidecode 관련 API를 호출하거나, 메모리 상의 특정 가상화 아티팩트(QEmu, VMware, VirtualBox 관련 문자열)를 검색하는 전형적인 정찰(Reconnaissance) 함수 호출 패턴을 탐지한다.

(3) MITRE ATT&CK - 공격 방법

T1003.007 - OS Credential Dumping: Proc Filesystem

공격자는 /proc 파일 시스템(proc filesystem)에서 자격 증명(Credentials)을 수집할 수 있다. proc 파일 시스템은 Linux 기반 시스템에서 커널의 데이터 구조와 상호작용하기 위한 가상(Pseudo) 파일 시스템으로, 프로세스의 메모리와 실행 상태 등 다양한 정보를 제공한다.

각 프로세스에는 /proc/<PID>/maps와 /proc/<PID>/mem 파일이 존재한다.

maps 파일은 해당 프로세스의 가상 메모리 주소 공간(Virtual Address Space)이 어떻게 매핑되어 있는지를 보여주며, mem 파일은 디버깅 목적으로 프로세스의 가상 메모리 내용에 접근할 수 있는 인터페이스를 제공한다.

공격자가 root 권한을 획득한 경우, 시스템에서 실행 중인 모든 프로세스의 메모리를 탐색하여 비밀번호, 인증 토큰, 해시(Hash)와 같은 자격 증명을 검색할 수 있다. 예를 들어, 정규표현식(Regex)을 이용하여 메모리 영역을 식별한 뒤, 메모리에 저장된 고정 문자열이나 캐시된 해시 값을 추출하는 방식이 사용될 수 있다.

루트 권한이 없는 경우에도 프로세스는 자신의 메모리 공간에는 접근할 수 있다. 일부 프로그램이나 서비스는 성능상의 이유로 비밀번호나 인증 정보를 평문(Plaintext) 형태로 메모리에 저장하는 경우가 있으며, 공격자는 이를 악용하여 자격 증명을 획득할 수 있다.

또한 웹 브라우저와 동일한 권한으로 실행되는 프로세스는 /proc/<PID>/maps와 /proc/<PID>/mem을 분석하여 웹사이트 로그인 정보와 관련된 메모리 패턴을 검색할 수 있다. 이를 통해 비밀번호 해시(Hash)나 평문 자격 증명이 저장된 메모리 영역을 찾아 탈취할 수 있다.

(4) Detection Strategy 분석

① AN1631

AN16431은 Linux 환경에서 다른 프로세스의 메모리 영역에 접근하려는 행위를 탐지하는 분석 기법이다.

공격자는 ptrace() 시스템 콜을 이용하여 대상 프로세스에 연결(Attach)하거나 메모리를 읽고(Read), 수정(Write)할 수 있다. 이러한 기능은 원래 gdb, strace와 같은 디버깅 도구에서 사용되지만, 공격자는 이를 악용하여 정상 프로세스의 메모리에서 비밀번호, 인증 토큰, API Key 등의 민감한 정보를 추출하거나 프로세스의 실행 흐름을 변경할 수 있다.

ptrace() 시스템 콜에서 사용되는 Request 값을 기반으로 프로세스 접근 행위를 탐지한다.

특히 PTRACE_ATTACH, PTRACE_PEEKDATA, PTRACE_PEEKTEXT, PTRACE_POKEADATA, PTRACE_POKETEXT와 같이 다른 프로세스에 대한 연결 또는 메모리 접근을 의미하는 Request를 Rule Engine의 탐지 조건으로 사용한다.

② Log Sources(Auditd에서 활용하는 로그)

ProcessAccess 이벤트를 탐지하기 위해 Linux Auditd Framework(Auditd)에서 생성되는 ptrace 관련 로그를 사용한다.

Auditd Record	활용 목적
SYSCALL	ptrace() 시스템 콜 발생 여부, PID, UID, EUID, Request 값(a0)을 수집한다.
PROCTITLE	원본 명령 문자열을 디버깅 및 분석에 활용한다.

Collector는 동일 Audit ID를 기준으로 위 Record를 하나의 Audit Event로 조립한다.

이후 Dispatcher는 SYSCALL.a0 값을 사람이 이해할 수 있는 PTRACE_ATTACH, PTRACE_PEEKDATA 등의 문자열로 변환하여 SystemEvent.Request로 전달한다.

③ Mutable Elements(Rule Engine에서 사용하는 조건 변수)

SystemEvent 필드	설명
Request	ptrace 요청 종류
ProcessName	ptrace를 수행한 프로세스 이름
CommandLine	Ptrace 실행 명령 및 인자
EUID	ptrace를 수행한 사용자 권한

(5) Rule

① AN1631 Rule 설계

AN1631 Rule은 ptrace() 시스템 콜을 이용하여 다른 프로세스에 접근하려는 행위를 탐지하기 위해 설계한다.

공격자는 ptrace()를 이용하여 프로세스에 연결하거나 메모리 내용을 읽고 수정할 수 있다. 이러한 기능은 정상적인 디버깅 도구에서도 사용되지만, 공격자가 자격 증명 탈취나 프로세스 조작을 목적으로 사용할 경우 보안상 중요한 탐지 대상이 된다.

프로젝트에서는 Auditd의 ptrace() 시스템 콜과 SYSCALL Record의 a0 값을 이용하여 ptrace Request 종류를 수집한다. 이후 Dispatcher는 a0 값을 SystemEvent.Request로 변환하고, Rule Engine은 해당 Request가 Attach, MemoryRead, MemoryWrite 그룹에 포함되는지를 검사하여 의심스러운 프로세스 접근 행위를 탐지한다.



② Rule 생성

```
{
  "rules": [
    {
      "rule_id": "DET0593_AN1631",
      "name": "Linux에서 /proc파일 시스템 접근을 통한 OS 자격 증명 덤프 탐지",
      "description": "공격자는 /proc 파일 시스템을 이용하여 프로세스의 민감한 메모리 영역에 접근하  
고, 자격 증명 정보를 추출하려는 행위를 탐지한다.",
      "event_type": "ProcessAccess",
      "syscall": "ptrace",

      "conditions": {
        "request": [
          "Attach",
          "MemoryRead",
          "MemoryWrite"
        ]
      }
    }
  ]
}
```

[7-3-6] PrivilegeEscalation Rule

(1) System Call

PrivilegeEscalation Event는 `setuid()`, `setgid()` System Call을 기반으로 생성한다.

① `setuid()`란

프로세스의 사용자 ID(UID)를 변경하여 실행 권한을 제어하는 시스템 콜이다.

주로 일반 사용자가 일시적으로 관리자(root) 권한을 획득하거나, 반대로 권한을 낮춰 보안을 유지할 때 사용된다.

사용자 ID의 3가지 종류

- 실제 사용자 ID(RUID, Real User ID): 프로세스를 실제로 소유하고 실행한 원래 사용자의 ID
- 유효 사용자 ID(EUID, Effective User ID): 커널이 파일에 접근하거나 작업을 수행할 때 권한을 확인하는 데 사용하는 ID
- 저장된 사용자 ID(SUID, Saved User ID): 변경된 EUID를 이전 값으로 복구하기 위해 보관해두는 ID

```
int setuid(uid_t uid);
```

- uid: 변경하고자 하는 대상 사용자의 숫자형 ID(예: 0은 root, 1000은 일반 사용자)

② `setgid()`란

유닉스 및 리눅스 시스템에서 현재 프로세스의 그룹 식별자(GroupID, GID)를 변경할 때 사용하는 시스템 콜이다. 이를 통해 프로세스는 실행 도중 권한을 그룹 레벨에서 제어할 수 있다.

주요 동작 원리

프로세스는 실행 주체에 따라 3가지 그룹 ID를 가진다. `setgid()`는 이 값들을 다음과 같이 변경한다.

- 일반 사용자(루트가 아닌 경우): 유효 GID(Effective GID, `egid`)만 지정한 GID로 변경한다.
- 슈퍼유저(root): 실제 GID(Real GID), 유효 GID(Effective GID), 그리고 저장된 GID(Saved `set-GID`)를 모두 지정한 GID로 변경한다.

```
int setgid(gid_t gid);
```

- `gid_t gid`: 변경하고자 하는 새로운 그룹 ID 번호

(2) MITRE ATT&CK – 탐지 전략

① Process Metadata

Process Metadata는 실행 중인 프로세스와 관련된 메타데이터를 의미하며, 환경 변수, 실행 파일 이름 (Image Name), 사용자 또는 소유자(User/Owner)와 같은 프로세스의 실행 환경 및 속성 정보를 포함한다.

② Detection Strategy

Sudo 및 Sudo 캐싱 오용(권한 상승) 탐지

- ID: DET0052
- 이름: Behavioral Detection Strategy for Abuse of Sudo and Sudo Caching
- MITRE ID: T1548.003(Abuse Elevation Control Mechanism: Sudo and Sudo Caching)
- 설명: Linux 환경에서 가장 흔하게 발생하는 권한 상승 패턴이다. 공격자가 사용자의 비밀번호를 알아낸 후 sudo 권한을 남용하거나, 이미 인증이 완료되어 비밀번호 없이 입력 가능한 Sudo 캐시 타임아웃 시간(기본 15분)을 노려 연속적으로 root 명령을 내리는 비정상적인 프로세스 행위를 탐지한다.

Setuid/Setgid 권한 남용 탐지

- ID: DET0110
- 이름: Setuid/Segid Privilege Abuse Detection(Linux/macOS)
- MITRE ID: T1548.001(Abuse Elevation Control Mechanism: Setuid and Setgid)
- 설명: 공격자가 일반 사용자 계정에서 root 권한을 얻기 위해 자주 쓰는 기법이다. 시스템 내에 SUID 비트가 잘못 설정된 정상 바이너리(예: find, vim 등)를 이용해 root 셸을 획득하거나, 자기가 만든 백도어 파일에 SUID 권한을 부여(chmod +s)하여 나중에 언제든지 root 권한으로 프로세스를 실행하려는 흐름을 잡아낸다.

동적 링커 하이재킹(LD_PRELOAD 백도어) 탐지

- ID: DET0435
- 이름: Detection Strategy for Hijack Execution Flow: Dynamic Linker Hijacking
- MITRE ID: T1544.006(Hijack Execution Flow: Dynamic Linker Hijacking)
- 설명: Linux 환경에서 방어 우회와 영속성 확보를 위해 매우 대중적으로 쓰이는 고도화된 기법이다. /etc/ld.so.preload 파일을 변조하거나 LD_PRELOAD 환경 변수를 악용하여 공유 라이브러리(..so)를 가로챈다. 이를 통해 일반 시스템 명령어(ls, ps 등)를 입력할 때 악성 코드가 먼저 실행되도록 유도하는 행위를 프로세스 환경 변수 모니터링을 통해 탐지한다.

프로세스 인자 변조(Overwritten Process Arguments) 탐지

- ID: DET0164
- 이름: Detection Strategy for Overwritten Process Arguments Masquerading
- MITRE ID: T1036.001(Masquerading: Overwritten Process Arguments)
- 설명: 공격자가 악성 스크립트나 마이너(Cryptojacker)를 실행할 때, 관리자의 눈과 모니터링 툴을 속이기 위해 프로세스 이름(인자)을 변조하는 기법이다. 예를 들어, 실제 실행된 바이너리는 악성코드인데, ps 명령어로 볼때는 정상적인 nginx나 kswapd0인 것처럼 메모리 상의 argv 값을 덮어쓰는 행위(Discrepancy)를 찾아낸다.

SSH 세션 하이재킹(Session Hijacking) 탐지

- ID: DET0256
- 이름: Detection Strategy for SSH Session Hijacking
- MITRE ID: T1563.001(Remote Service Session Hijacking: SSH Session Hijacking)
- 설명: 이미 내부망에 침투한 공격자가 다른 내부 Linux 서버로 수평 이동(Lateral Movement)할 때 자주 쓴다. 피해자 시스템에 연결되어 있는 기존 SSH 에이전트 소켓 파일(SSH_AUTH_SOCK)의 권한을 탈취하여, 비밀번호나 개인 키 없이도 기존 연결된 세션을 그대로 가로채 원격 서버에 무단 로그인하는 행위를 프로세스 관계 및 환경 변수 조회를 통해 탐지한다.

(3) MITRE ATT&CK - 공격 방법

T1548.001 - Abuse Elevation Control Mechanism: Setuid and Setgid

공격자는 Setuid(Set User ID) 또는 Setgid(Set Group ID) 비트가 설정된 프로그램을 악용하여, 다른 사용자 또는 더 높은 권한을 가진 사용자(또는 그룹)의 권한으로 코드를 실행할 수 있다.

Linux와 macOS에서는 실행 파일(Binary)에 Setuid 또는 Setgid 비트가 설정되어 있으면, 해당 프로그램은 실행한 사용자의 권한이 아니라 파일 소유자(User) 또는 소유 그룹(Group)의 권한으로 실행된다.

일반적으로 프로그램은 현재 사용자의 권한으로 실행되지만, 일부 프로그램은 정상적인 기능 수행을 위해 관리자 권한과 같은 높은 권한이 필요하므로 Setuid 또는 Setgid가 사용된다.

공격자는 sudoers 파일에 권한을 추가하는 대신, 자신이 소유한 실행 파일에 Setuid 또는 Setgid 비트를 설정하여 권한 상승을 수행할 수 있다. 이러한 비트는 chmod 명령을 이용하여 설정할 수 있으며, 예를 들어, 다음과 같은 명령을 사용할 수 있다.

- Setuid 설정
 - chmod 4777 <file>
 - chmod u+s <file>
- Setgid 설정
 - chmod 2775 <file>
 - chmod g+s <file>

공격자는 자신의 악성 프로그램에 이러한 비트를 설정하여 향후에도 관리자 권한으로 실행될 수 있도록 영속성(Persistence)과 권한 상승(Privilege Escalation)을 확보할 수 있다.

이러한 기법은 Shell Escape나 제한된 실행 환경을 우회하기 위한 공격 과정에서 자주 사용된다.

또한 공격자는 이미 시스템에 존재하는 Setuid 또는 Setgid 비트가 설정된 취약한 실행 파일을 탐색하여 이를 악용할 수도 있다. ls -l 명령으로 파일 권한으로 확인하면 실행 권한(x) 대신 s로 표시되며, find 명령을 사용하여 이러한 파일을 검색할 수 있다.

예를 들어,

- find / -perm +4000 2>/dev/null
 - Setuid 비트가 설정된 파일 검색
- find / -perm +20000 2>/dev/null
 - Setgid 비트가 설정된 파일 검색

공격자는 이렇게 발견한 실행 파일의 취약점을 악용하여 권한 상승을 수행할 수 있다.

(4) Detection Strategy 분석

① AN0307

AN0307은 Linux/macOS 환경에서 Setuid 또는 Setgid 권한이 악용되어 프로세스가 일반 사용자 권한이 아닌 더 높은 권한으로 실행되는 행위를 탐지하기 위한 분석 기법이다.

Setuid 또는 Setgid 비트가 설정된 실행 파일은 실행 사용자의 권한이 아니라 파일 소유자 또는 소유 그룹의 권한으로 실행될 수 있다. 이 기능은 정상적으로는 시스템 관리 도구나 인증 관련 프로그램에서 사용되지만, 공격자는 이를 악용하여 일반 사용자 권한에서 root 권한을 획득하거나, 자신이 만든 악성 파일을 이후에도 높은 권한으로 실행되도록 만들 수 있다.

1차 구현 범위에서 chmod를 통한 Setuid/Setgid 비트 설정 행위와 실행 파일 권한 변환까지는 연계 분석하지 않고, setuid() 또는 setgid() 시스템 콜이 호출된 이후 유효 사용자 또는 유효 그룹이 root 권한으로 변경되는 행위를 탐지 대상으로 선정한다.

② Log Sources(Auditd에서 활용하는 로그)

PrivilegeEscalation 이벤트를 탐지하기 위해 Linux Audit Framework(Auditd)에서 생성되는 setuid, setgid 관련 로그를 사용한다.

Auditd Record	활용 목적
SYSCALL	setuid() 또는 setgid() 시스템 콜 발생 여부와 UID, EUID, GUID, EGID 값을 수집한다.
PROCTITLE	원본 명령 문자열을 디버깅 및 분석에 활용한다.

Collector는 동일 Audit ID를 기준으로 위 Record를 하나의 Audit Event로 조립한다.

이후 Dispatcher는 SYSCALL.euid, SYSCALL.egid 값을 SystemEvent.EUID SystemEvent.EGID로 변환하여 Rule Engine으로 전달한다.

③ Mutable Elements(Rule Engine에서 사용하는 조건 변수)

SystemEvent 필드	설명
EUID	프로세스가 실제로 권한 검사를 받을 때 사용되는 유효 사용자 ID
EGID	프로세스가 실제로 권한 검사를 받을 때 사용되는 유효 그룹 ID
ProcessName	권한 변경을 수행한 프로세스 이름
CommandLine	권한 변경과 관련된 명령 및 인자

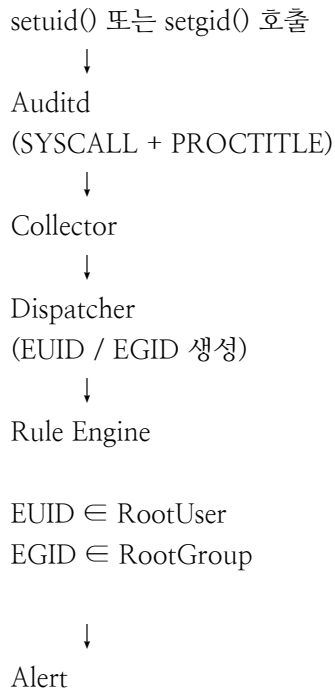
(5) Rule

① AN0307 Rule 설계

AN0307 Rule은 `setuid()` 또는 `setgid()` 시스템 콜을 프로세스의 유효 권한이 root 권한으로 변경되는 행위를 탐지하기 위해 설계한다.

Linux에서는 실제 사용자 ID(UID)와 유효 사용자 ID(EUID)가 구분된다. 커널은 파일 접근이나 구멍이 필요한 작업을 수행할 때 주로 EUID를 기준으로 권한을 판단한다. 따라서 일반 사용자로 실행된 프로세스가 `setuid()` 또는 `setgid()` 호출 이후 EUID 또는 EGID가 root(0)로 변경되는 경우, 권한 상승 가능성이 있는 행위로 볼 수 있다.

Auditd의 `setuid()` 및 `setgid()` 시스템 콜과 SYSCALL Record를 이용하여 EUID, EGID 값을 수집한다. 이후 Dispatcher는 이를 `SystemEvent.EUID`, `SystemEvent.EGID`로 변환하고, Rule Engine은 해당 값이 RootUser 또는 RootGroup 그룹에 포함되는지를 검사하여 권한 상승 행위를 탐지한다.



② Rule 생성

```
{
  "rules": [
    {
      "rule_id": "DET0110_AN0307",
      "name": "Setuid/Segid 권한 남용 탐지",
      "description": "Chmod 명령으로 Setuid 또는 Setgid 비트를 설정한 이후, 실제 사용자 ID(UID)와 유효 사용자(EUID)가 서로 다른 권한으로 프로세스가 실행되는 행위를 연계 분석하여 탐지한다.",
      "log_source": [
        "auditd:setuid",
        "auditd:setgid"
      ],
      "conditions": {
        "euid": "RootUser",
        "egid": "RootGroup"
      }
    }
  ]
}
```

[7-3-7] Persistence Rule - 1

(1) MITRE ATT&CK - 탐지 전략

① File Modification

파일 수정(File Modification)은 파일의 내용(Content), 메타데이터(Metadata), 접근 권한(Permissions), 속성(Attributes) 등이 변경되는 행위를 의미한다. 이러한 변경은 소프트웨어 업데이트와 같은 정상적인 활동일 수도 있지만, 파일 변조(Tampering), 랜섬웨어 감염, 또는 공격자의 악의적인 수정과 같은 비정상적인 행위일 수도 있다.

대표적인 예시는 다음과 같다.

- 내용 수정(Content Modifications): Linux의 /etc/ssh/sshd_config 또는 Windows의 C:\Windows\System32\drivers\etc\hosts 와 같은 설정 파일의 내용을 변경하는 행위
- 권한 변경(Permission Changes): Linux에서 파일 권한을 644에서 777로 변경하거나, Windows에서 NTFS 파일 권한을 변경하여 접근 권한을 확대하는 행위
- 속성 변경(Attribute Modifications): Windows에서 파일의 속성을 숨김(Hidden), 읽기 전용(Read-only), 시스템(System) 등으로 변경하는 행위
- 타임스탬프 조작(Timestamp Manipulation): Linux의 touch 명령이나 Windows의 타임스톰핑(TIMESTOMPING) 도구를 이용하여 파일의 생성 시간 또는 수정 시간을 조작하는 행위
- 소프트웨어 및 시스템 파일 변경(Software or System File Changes): boot.ini, 커널 모듈(Kernel Module), 애플리케이션 실행 파일(Binary) 등 시스템 파일을 수정하는 행위

② Detection Strategy

리눅스 감사 시스템(Auditd) 비활성화 및 변조 탐지

- ID: DET0062
- 이름: Detection Strategy for Disable or Modify Linux Audit System Log
- MITRE ID: T1685.004(Indicator Removal: Disable or Modify Tools)
- 설명: Linux 환경에서 모니터링을 우회하려는 공격자들이 가장 먼저 시도하는 행위이다. 타겟 시스템의 커널 이벤트를 로깅하는 auditd 서비스를 강제로 중지(systemctl stop auditd)하거나, 감사 규칙 파일(/etc/audit/audit.rules)을 무단으로 수정, 삭제하여 탐지를 무력화하려는 시도를 잡아내는 최우선 방어 전략이다.

PAM(플러그형 인증 모듈) 백도어 변조 탐지

- ID: DET0454
- 이름: Detect Malicious Modification of Pluggable Authentication Modules (PAM)
- MITRE ID: T1556.003(Modify Authentication Process: Pluggable Authentication Modules)
- 설명: Linux 시스템의 로그인 및 인증 체계를 장악하기 위한 고도화된 영속성 기법이다. 공격자가 /etc/pam.d/ 디렉토리 내의 인증 설정 파일을 변조하거나 악성 PAM 모듈(.so)을 삽입하여, 특정 마스터 비밀번호만 입력하면 어떤 계정으로든 로그인이 가능하게 만드는 '스켈레톤 키'형태의 백도어 행위를 탐지한다.

SSH 공인 키 주입(Authorized Keys) 탐지

- ID: DET0126
- 이름: Detection Strategy for SSH Key Injection in Authorized Keys
- MITRE ID: T1098.004(Account Manipulation: SSH Authorized Keys)
- 설명: 공격자가 비밀번호 변경 없이 언제든지 원격에서 다시 접속할 수 있도록 흔히 쓰는 방식이다. 특정 사용자 혹은 root 계정의 ~/.ssh/authorized_keys 파일에 공격자의 Public Key를 몰래 추가하는 행위를 감시한다. 시스템 관리 도구가 아닌 비정상적인 프로세스(예: 웹 서버 권한 등)가 이 파일을 수정할 때 즉각 탐지한다.

리눅스 파일 권한 및 속성 변조(타임스톰핑/권한 오용) 탐지

- ID: DET0351
- 이름: Unix-like File Permission Manipulation Behavior/ Chain Detection Strategy
- MITRE ID: T1222.002(File and Directory Permissions Modification: Linux and macOS Permissions)
- 설명: 공격자가 악성 스크립트를 실행 가능한 상태로 만들거나 타겟 파일을 탈취하기 위해 수행한다. 웹 루트 디렉터리나 주요 설정 권한을 chmod 777 등으로 과도하게 변경하는 행위, 혹은 EDR의 타임라인 분석을 교란하기 위해 touch -r 등을 이용해 파일 생성/수정 시간을 과거로 조작하는 타임스톰핑(TIMESTOMPING) 행위를 추적한다.

Udev 규칙(Hardware Event Trigger)을 이용한 우회 실행 탐지

- ID: DET0375
- 이름: Detection Strategy for T1546.017 - Udev Rules(Linux)
- MITRE ID: T1546.017(Event Triggered Execution: Udev Rules)
- 설명: 크론탭(Crontab)이나 Systemd 서비스 등록보다 한 단계 더 숨겨진 영속성 확보 기법이다. Linux에서 디바이스 장치가 연결되거나 시스템 이벤트가 발생할 때 실행되는 udev 규칙 설정 파일(/etc/udev/rules.d/)을 변조하여, 특정 하드웨어 이벤트 트리거 시 악성코드가 백그라운드에서 자동으로 실행되도록 만드는 행위를 감시한다.

(3) MITRE ATT&CK - 공격 방법

T1546.017 - Event Triggered Execution: Udev Rules

공격자는 Udev 규칙(Udev Rules)을 악용하여 악성 코드를 자동 실행함으로써 시스템 영속성(Persistence)을 유지할 수 있다.

Udev는 Linux 커널의 장치 관리자(Device Manager)로, 장치 노드(Device Node)를 동적으로 관리하고 /dev 디렉터리의 가상 장치 파일(Pseudo-device File)에 대한 접근을 처리한다. 또한 하드디스크나 키보드와 같은 외부 장치의 연결 또는 제거와 같은 하드웨어 이벤트에 반응하여 필요한 작업을 수행한다.

Udev는 규칙 파일을 사용하여 동작을 정의한다. 규칙 파일에는 매치 키(Match Key)를 통해 특정 하드웨어 이벤트가 만족해야 하는 조건을 지정하고, 액션 키(Action Key)를 통해 조건이 충족되었을 때 수행할 작업을 정의한다.

Udev 규칙 파일은 다음과 같은 디렉터리에 저장되며, 생성, 수정, 삭제를 위해서는 Root 권한이 필요하다.

- /etc/udev/rules.d/
- /run/udev/rules.d/
- /usr/lib/udev/rules.d/
- /usr/local/lib/udev/rules.d/
- /lib/udev/rules.d/

또한 규칙의 우선순위는 저장된 디렉터리와 파일 이름의 숫자 접두사(Prefix)에 의해 결정된다.

공격자는 이러한 Udev 하위 시스템을 악용하여 규칙 파일을 새로 생성하거나 기존 규칙을 수정함으로써 악성 코드를 자동 실행할 수 있다. 예를 들어, /dev/random과 같은 가상 장치 파일에 애플리케이션이 접근할 때마다 공격자가 지정한 실행 파일(Binary)이 실행되도록 규칙을 설정할 수 있다.

비록 Udev는 짧은 작업만 수행하도록 설계되어 있으며, systemd-udevd의 샌드박스(Sandbox) 환경으로 인해 네트워크 및 파일 시스템 접근이 제한되지만, 공격자는 RUN+= 액션 키에 스크립트 명령을 등록하여 악성 프로세스를 백그라운드에서 분리(detech)하여 실행함으로써 이러한 제한을 우회할 수 있다. 이를 통해 악성 코드가 지속적으로 실행되도록 하여 시스템에 대한 영속성(Persistence)을 확보할 수 있다.

(4) Detection Strategy 분석

① AN1056

AN1056은 Linux의 Udev Rules를 악용하여 시스템 이벤트 발생 시 악성 프로그램이 자동으로 실행되도록 설정하는 영속성 확보 기법을 탐지하기 위한 분석 기법이다.

Udev는 Linux의 장치(Device) 관리 시스템으로, 저장 장치나 USB와 같은 하드웨어 이벤트가 발생하면 등록된 규칙(Udev Rules)에 따라 지정된 작업을 수행한다.

공격자는 이러한 특성을 악용하여 /etc/udev/rules.d/, /lib/udev/rules.d/ 등의 규칙 디렉터리에 새로운 Rule 파일을 생성하거나 기존 Rule을 수정하여 특정 이벤트 발생 시 악성 프로그램이 자동으로 실행되도록 만들 수 있다.

Udev Rule 내부의 RUN+= 키 분석이나 systemd-udevd 프로세스와의 행위 연계는 구현 범위를 벗어나므로, Udev Rule 디렉터리에서 규칙 파일이 생성되는 행위 자체를 영속성 확보의 징후로 탐지하도록 설계한다.

② Log Sources(Auditd에서 활용하는 로그)

PrivilegeEscalation 이벤트를 탐지하기 위해 Linux Audit Framework(Auditd)에서 생성되는 setuid, setgid 관련 로그를 사용한다.

Auditd Record	활용 목적
SYSCALL	creat() 시스템 콜 발생 여부 확인
PATH	생성된 파일의 경로 확인
CWD	파일 생성 당시 작업 디렉터리 확인

Collector는 Audit ID를 기준으로 위 Record를 하나의 FileCreate 이벤트로 조립한다.
 이후 Dispatcher로 생성된 파일 경로를 SystemEvent.TargetFile 또는 SystemEvent.PathName으로 변환하여 Rule Engine으로 전달한다.

③ Mutable Elements(Rule Engine에서 사용하는 조건 변수)

필드	설명
pathname	생성된 파일의 전체 경로
process_name	파일을 생성한 프로세스
command_line	생성 시 사용된 명령

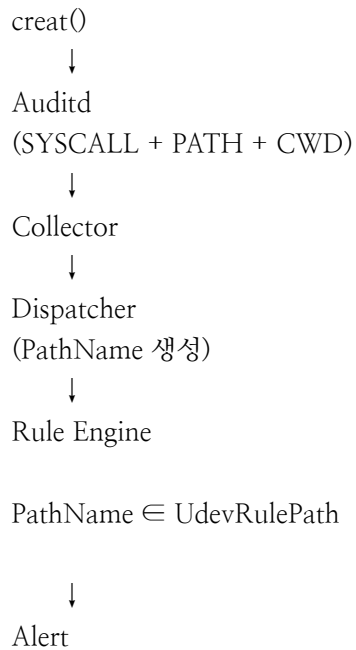
(5) Rule

① AN1056 Rule 설계

AN1056 Rule은 Udev Rule 디렉터리에서 새로운 Rule 파일이 생성되는 행위를 탐지하기 위해 설계한다. Linux에서는 Udev Rule이 /etc/udev/rules.d/, /lib/udev/rules.d/, /usr/lib/udev/rules.d/등에 저장된다.

공격자는 이러한 디렉터리에 새로운 Rule을 추가하여 장치 이벤트 발생 시 악성 프로그램이 자동 실행 되도록 설정할 수 있다.

Auditd의 FileCreate 이벤트를 이용하여 생성된 파일의 경로를 수집하고, Rule Engine은 생성된 파일 Udev Rule 디렉터리에 위치하는지를 검사하여 연속성 확보 시도를 탐지한다.



② Rule 생성

```
{
  "rules": [

    {
      "rule_id": "DET0375_AN1056",
      "name": "Udev 규칙 탐지 전략",
      "description": "Udev Rule 디렉터리에서 Rule 파일이 생성되는 행위를 탐지한다.",
      "event_type": "Persistence",
      "syscall": "creat",

      "conditions": {
        "pathname": "UdevRulePath"
      }
    }
  ]
}
```

[7-3-7] Persistence Rule - 2

(1) MITRE ATT&CK - 탐지 전략

① Command Execution

명령 실행(Command Execution)은 운영체제 또는 애플리케이션에서 텍스트 기반 명령어(셸 명령, Cmdlet, 스크립트 등)가 실행되는 행위를 모니터링하고 기록하는 것을 의미한다.

이러한 명령에는 인자(Arguments)나 매개변수(Parameters)가 포함될 수 있으며, 일반적으로 cmd.exe, bash, zsh, PowerShell과 같은 명령 인터프리터 또는 프로그램을 통해 실행된다.

대표적인 예시는 다음과 같다.

Windows 명령 프롬프트(Command Prompt)

- dir: 현재 디렉터리의 파일 및 폴더 목록을 출력한다.
- net user: 사용자 계정을 조회하거나 생성, 수정하는 명령이다.
- tasklist: 현재 실행 중인 프로세스 목록을 조회한다.

PowerShell

- Get-Process: 시스템에서 실행 중인 프로세스를 조회한다.
- Set-ExecutionPolicy: PowerShell 스크립트를 실행 정책을 변경한다.
- Invoke-WebRequest: 원격 서버의 리소스를 다운로드하거나 HTTP 요청을 수행한다.

Linux Shell

- ls: 디렉터리에 존재하는 파일 및 디렉터리 목록을 출력한다.
- cat /etc/passwd: 사용자 계정 정보가 저장된 /etc/passwd 파일의 내용을 조회한다.
- curl <http://malicious-site.com>: 원격 서버에 HTTP 요청을 보내고 데이터를 가져온다. 악성 서버와의 통신에도 악용될 수 있다.

컨테이너 환경(Container Environments)

- Docker exec: 실행 중인 Docker 컨테이너 내부에서 명령을 실행한다.
- Kubectl exec: Kubernetes Pod 내부에서 명령을 실행한다.

macOS 터미널

- open: 파일이나 URL을 실행하거나 연다.
- dscl . -list /Users: 시스템에 등록된 사용자 계정 목록을 조회한다.
- osascript -e: AppleScript 명령을 실행한다.

② Detection Strategy

Unix 셸 실행 및 명령행 오용(CLI Abuse) 탐지

- ID: DET0384
- 이름: Behavioral Detection of Unix Shell Execution
- MITRE ID: T1059.004(Command and Scripting Interpreter: Unix Shell)
- 설명: Linux 서버 침해 사고 발생 시 공격자의 99%는 명령어 인터페이스(bash, sh)를 통해 제어권을 얻는다. 일반적인 관리 패턴과 다른 비저상적인 부모 프로세스(예: 웹 서버, DB 프로세스 등)가 셸을 실행하거나, /tmp, /dev/shm 등 쓰기 권한이 열려 있는 임시 디렉터리에서 스크립트가 실행되는 행위를 잡아내는 가장 기본적이고 강력한 전략이다.

Linux 감사 시스템(Auditd) 우회 및 방어 무력화 탐지

- ID: DET0062
- 이름: Detection Strategy for Disable or Modify Linux Audit System log
- MITRE ID: T1685.004(Indicator Removal: Disable or Modify Tools)
- 설명: 권한을 장악한 공격자가 자신의 행위가 중앙 로그 서버(SIEM)로 전송되는 것을 막기 위해 가장 먼저 수행하는 방어 우회 기법이다. Linux 커널 로깅 백엔드인 auditd 서비스를 강제로 중지(systemctl stop auditd)하거나 관련 설정 규칙 파일을 변조, 삭제하는 행위를 추적하여 탐지 시스템의 무결성을 지킨다.

SSH 공인 키 주입을 통한 무단 계정 조작 탐지

- ID: DET0126
- 이름: Detection Strategy for SSH Key Injection in Authorized Keys
- MITRE ID: T1098.004(Account Manipulation: SSH Authorized Keys)
- 설명: 백도어 파일 설치보다 탐지가 까다롭고 공격자들이 선호하는 지속성 확보 방식이다. 정상적인 계정 관리 절차(엔서블 등의 형상관리 툴)가 아닌, 웹 권한이나 일반 사용자 권한의 프로세스가 ~/.ssh/authorized_keys 파일을 수정하여 공격자의 공개 키를 삽입하는 행위를 실시간으로 감시한다.

시스템 및 네트워크 정보 정찰(Discovery) 행위 탐지

- ID: DET0525
- 이름: System Discovery via Native and Remote Utilities
- MITRE ID: T1082(System Information Discovery)
- 설명: 공격자가 서버에 들어온 직후 내부망 확장을 위해 주변 환경을 정찰하는 단계이다. 단시간 내에 uname -a, cat /etc/issue, ifconfig, netstat -antp, route 등 시스템 사양과 네트워크 연결 상태를 확인하는 기본 명령어들이 연속적으로 묶여서 실행되는 특유의 '정찰 행위 체인'을 탐지한다.

도커 및 쿠버네티스 컨테이너 환경 관리 명령 오용 탐지

- ID: DET0083
- 이름: Container CLI and API Abuse via Docker/Kubernetes(T1059.013)
- MITRE ID: T1059.013(Command and Scripting Interpreter: Container Cloud CLI)
- 설명: 현대의 많은 Linux 서버가 컨테이너 환경(Docker, K8s)으로 운영되는 점을 겨냥한 전략이다. 침투한 공격자가 컨테이너 내부에서 호스트 OS로 탈출(Escape)하거나 다른 파드(Pod)를 장악하기 위해 docker exec, kubectl exec 등의 명령어로 권한 상승을 시도하거나 API를 오용하는 행위를 잡아낸다.

(3) MITRE ATT&CK - 공격 방법

T1098.004 - Account Manipulation: SSH Authorized Keys

공격자는 SSH의 authorized_keys 파일을 수정하여 피해 시스템에 대한 영속성을 확보할 수 있다. Linux 배포판, macOS, 그리고 ESXi 하이퍼바이저는 원격 관리 시 SSH의 공개키(Public Key) 기반 인증을 널리 사용한다. SSH의 authorized_keys 파일에는 해당 사용자 계정으로 로그인할 수 있는 공개키가 저장되며, 일반적으로 다음 위치에 존재한다.

- Linux/macOS: <사용자 홈 디렉터리>/ssh/authorized_keys
 - ESXi: /etc/ssh/keys-username/authorized_keys
- 또한 사용자는 SSH 설정 파일(/etc/ssh/sshd_config)을 수정하여 공개키 인증과 루트 계정 로그인을 활성화할 수 있다. 예를 들어,
- PubkeyAuthentication yes
 - RSAAuthentication yes
 - PermitRootLogin yes

와 같은 설정을 적용하여 공개키 인증과 Root 계정의 SSH 로그인이 가능해진다.

공격자는 스크립트나 셸 명령을 이용해 authorized_keys 파일에 자신이 보유한 공개키(Public Key)를 추가함으로써, 해당 공개키와 짝을 이루는 개인키(Private Key)만 가지고 있으면 언제든지 정상 사용자 계정으로 SSH에 로그인 할 수 있다. 이를 통해 비밀번호를 변경하지 않고도 지속적으로 시스템에 접근할 수 있어 영속성을 확보할 수 있다.

클라우드 환경에서도 동일한 공격이 가능하다. 예를 들어,

- Google Cloud에서는 add-metadata 명령을 이용해 가상 머신 사용자 계정에 SSH 공개키를 추가할 수 있으며,
- Microsoft Azure에서는 API에 PATCH 요청을 보내 authorized_keys 파일을 수정할 수 있다.

이러한 방법을 통해 공격자는 원격에서 지속적으로 시스템에 접속할 수 있으며, 더 높은 권한을 가진 사용자 계정에 공개키를 추가할 경우 권한 상승도 가능하다.

또한 authorized_keys 방식은 서버뿐만 아니라 네트워크 장비에서도 사용될 수 있으며, 예를 들어, ip ssh pubkey-chain 명령을 이용해 SSH 공개키를 등록할 수 있다.

(4) Detection Strategy 분석

① AN0350

AN0350은 공격자가 authorized_keys 파일에 자신의 SSH 공개키를 추가하여 비밀번호 없이 시스템에 다시 접속할 수 있도록 영속성을 확보하는 행위를 탐지하기 위한 분석 기법이다.

Linux에서는 SSH 공개키 인증을 위해 사용자별 ~/.ssh/authorized_keys 파일을 사용한다. 공격자는 이 파일을 수정하여 자신의 공개키를 추가하면 이후에는 비밀번호 없이 SSH 로그인에 가능해진다. authorized_keys 파일 자체에 대한 접근 또는 생성 행위를 영속성 확보의 핵심 징후로 판단하여 탐지한다.

② Log Sources(Auditd에서 활용하는 로그)

Auditd Record	활용 목적
SYSCALL	파일 생성 또는 수정 시스템 콜 확인
PATH	접근한 파일 경로 확인
CWD	작업 디렉터리 확인
EXECVE	실행된 명령 및 인자 확인

Collector는 위 Record를 하나의 Persistence이벤트로 조립하고, Dispatcher는 파일 경로와 명령행을 Rule Engine으로 전달한다.

③ Mutable Elements(Rule Engine에서 사용하는 조건 변수)

필드	설명
pathname	접근한 authorized_keys 파일 경로
process_name	파일을 수정한 프로세스
command_line	실행된 명령행

(5) Rule

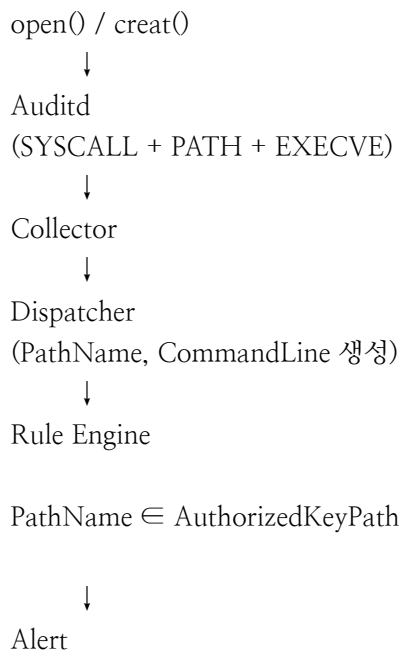
① AN0350 Rule 설계

AN0350 Rule은 공격자가 SSH 공개키를 등록하기 위해 authorized_keys 파일을 수정하는 행위를 탐지하기 위해 설계한다.

Linux에서는 사용자별 공개키 인증 파일이 ~/.ssh/authorized_keys 에 저장된다.

공격자는 자신의 공개키를 이 파일에 추가하면 이후에는 비밀번호를 입력하지 않고도 원격에서 SSH로 접속할 수 있다.

Auditd가 기록하는 파일 생성 및 접근 이벤트를 이용하여 authorized_keys 파일에 대한 접근을 탐지하며, Rule Engine은 생성되거나 수정된 파일의 경로가 SSH 공개키 저장 위치인지 검사하여 연속성 확보 시도를 탐지한다.



② Rule 생성

```
{
  "rules": [

    {
      "rule_id": "DET0126_AN0350",
      "name": "Authorized Keys 내 SSH 키 주입 탐지 전략",
      "description": "authorized_keys 파일에 대한 생성 또는 수정 행위를 탐지한다.",
      "event_type": "Persistence",

      "conditions": {
        "pathname": "AuthorizedKeyPath"
      }
    }

  ]
}
```

